

# Software Design and Architecture

## Lecture-3

By: Usama Musharaf

A solid blue wave-like shape that spans the width of the slide, positioned at the bottom.

# Software Architecture

(High Level Design)

# Software Architecture

Software architecture is the high-level structure and design of a software system. It defines how a system is organized, how its components interact, and the key decisions that guide its development and evolution.

## **Good architecture helps with:**

- Scalability – Can the system handle more users/data?
- Maintainability – Is it easy to modify and extend?
- Performance – Is it fast and efficient?
- Reliability & availability – Does it tolerate failures?
- Security – Are data and services protected?

## **Poor architecture leads to:**

- Tight coupling
- Spaghetti code
- Difficult debugging

# Key Elements of Software Architecture

## 1. Components

Major building blocks of the system.

Examples:

UI layer, Business logic, Database

## 2. Connectors

How components communicate:

REST APIs

Message queues

Function calls

## 3. Constraints

Rules that must be followed:

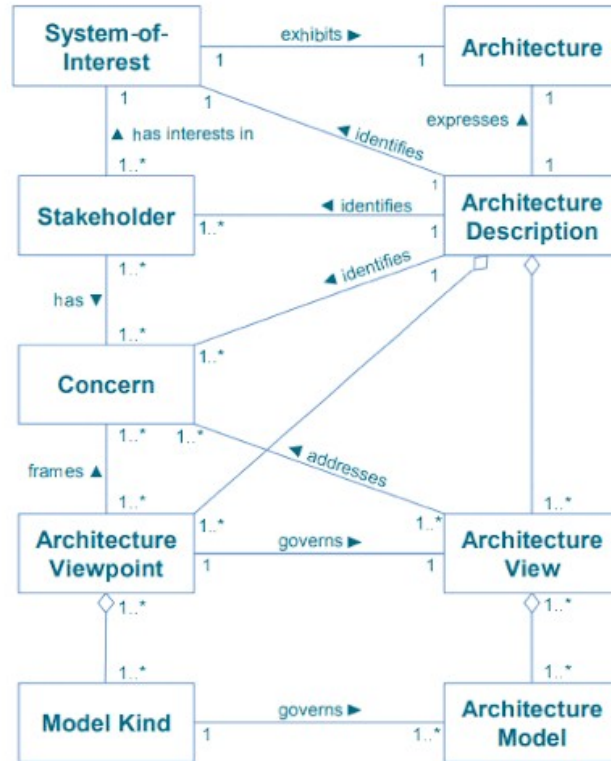
Use of specific technologies

Security policies

Performance requirements

# Conceptual Model of an Architecture Description

# Conceptual Model of an Architecture Description



# Views and Viewpoints

As per IEEE definition,

Software architecture descriptions are commonly organized into views,

Each view addresses a set of system concerns, following the conventions of its viewpoint.

Viewpoint- A position or direction from which something is observed or considered;

View – Details or full specification considered from that viewpoint

# Views and Viewpoints

## **View:**

A representation of the system from a particular perspective.

Example:

- Database view
- Deployment view
- Process View
- Network View



# Views and Viewpoints

## **Viewpoint:**

The set of concerns, rules, and criteria used to construct and analyze a view.

### **Database View Shows:**

Tables, Schemas, Connections, Data flows

### **Viewpoint:**

Security & Performance Viewpoint

### **Defines:**

- What security properties to check
- What performance metrics to assess
- What risks to identify

# 4 + 1 View Model

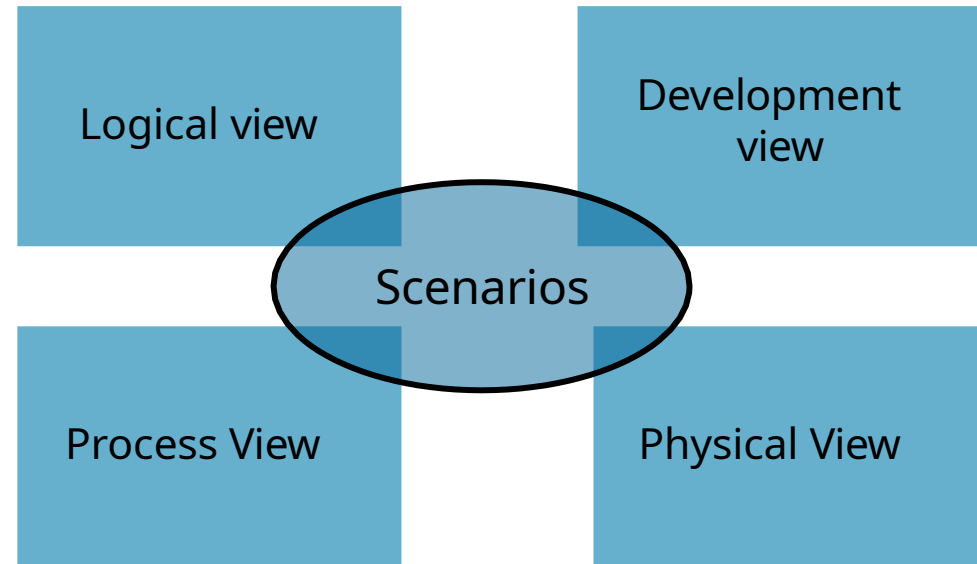
# 4 + 1 View Model

The 4+1 view model was originally introduced by Philippe Kruchten (Kruchten, 1995).

The model provides four essential views:

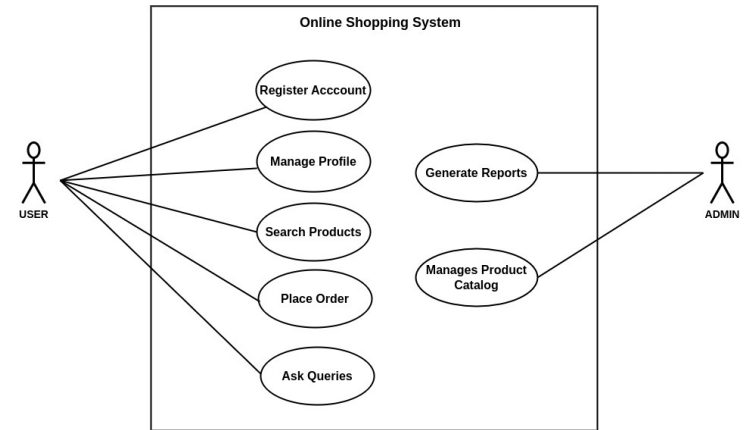
- the logical view,
- the process view,
- the physical view,
- the development view

and fifth is the scenario view



# Use Case View (scenario view)

- They use case view illustrates the functionality of the system.
- Using use case we can capture the goals of the user or what the user expects from the system.
- In UML, Use Cases can be created through use case diagrams or use case descriptions.
- Use cases can be created by analysts' architects or even by the users.



# Process View

The process view focuses on the dynamic aspects of the system, i.e., its execution time behavior.

This view maps functions, activities, and interactions onto runtime implementation.

So using process view we can understand the overall functioning of the system

## Process View

- Activity Diagram
- Sequence (optional)
- State (optional)

# Development View

The development view describes the modules and the components of the system.

This might include packages or libraries.

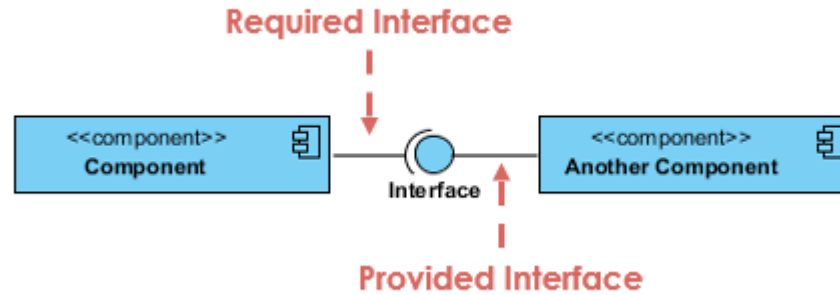
It gives a high-level view of the architecture of the system.

## Development View

- Components diagram
- Composite structure diagram
- Package diagram

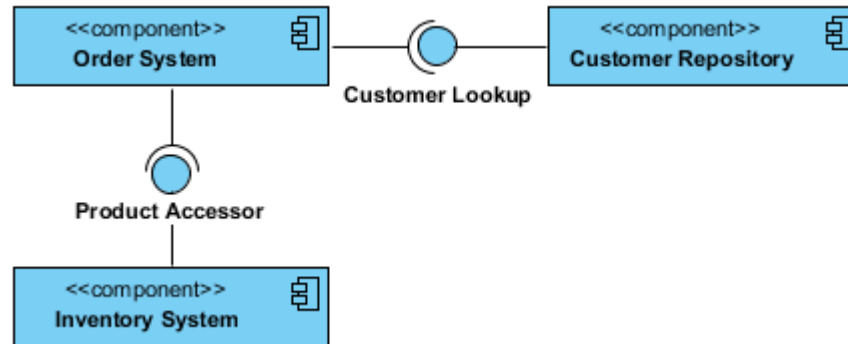
# Development View

## Component Diagram



# Development View

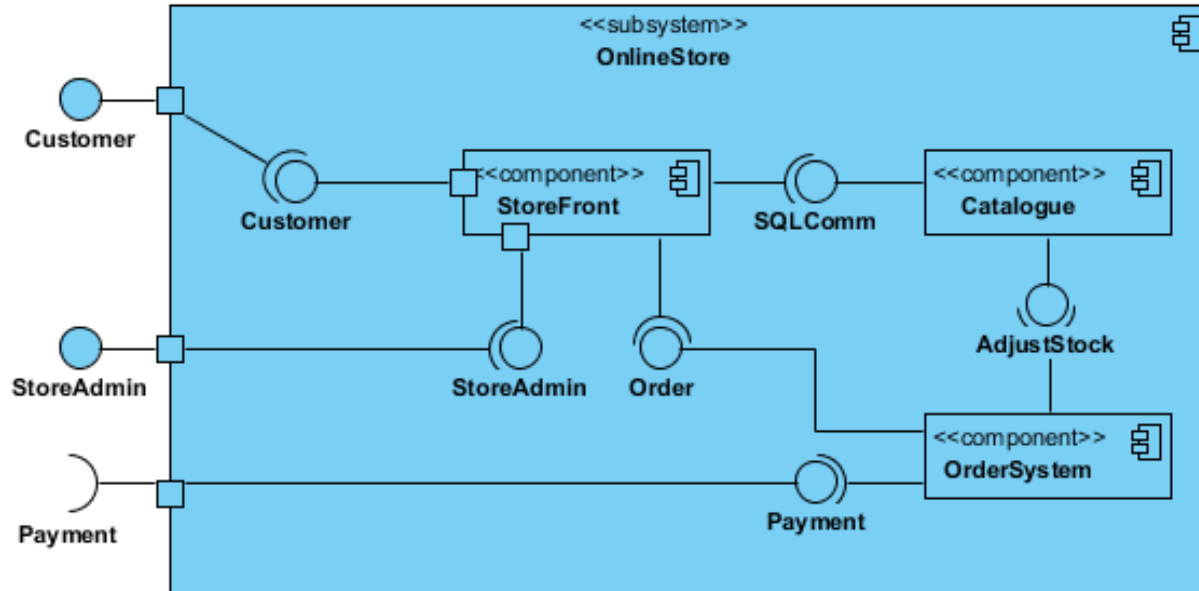
## Component Diagram





# Development View

## Subsystems (logical grouping of components)



# Development View

## Composite Structure Diagram

A Composite Structure Diagram is a UML diagram that shows the internal structure of a classifier and how its parts collaborate to provide its behavior.

### What is a “classifier”?

In UML, a classifier can be:

Class

Component

Subsystem

So a composite structure diagram can describe the inside of any of these.

### What it shows

A composite structure diagram shows:

Parts – internal components

Ports – interaction points with the outside

Connectors – links between parts

Interfaces – provided/required services

Collaborations – how parts work together

# Development View

## Composite Structure Diagram

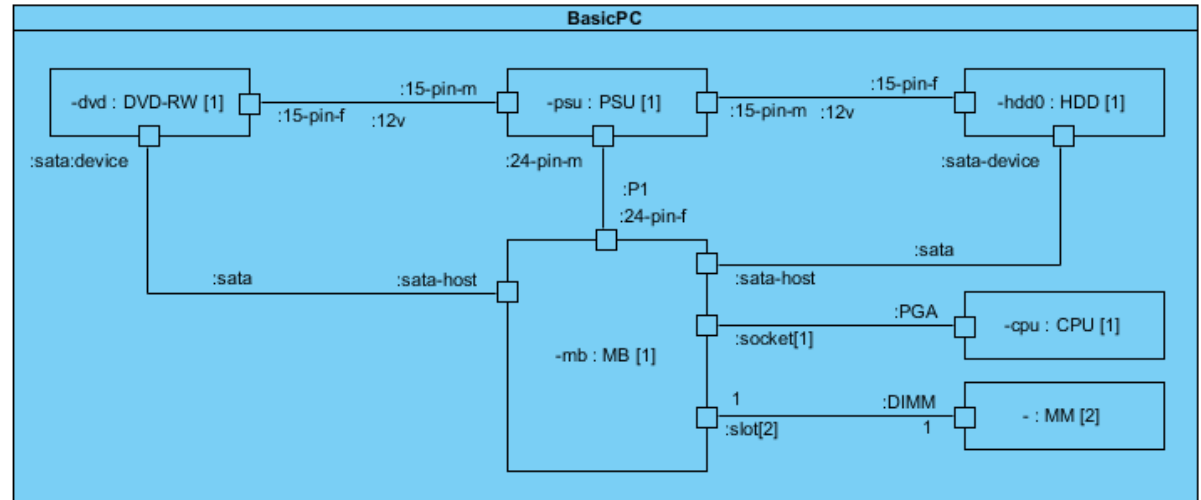
Power Supply Unit (PSU)

Hard Disk Drive (HDD)

Mainboard (MB)

Optical Drive (DVD-RW)

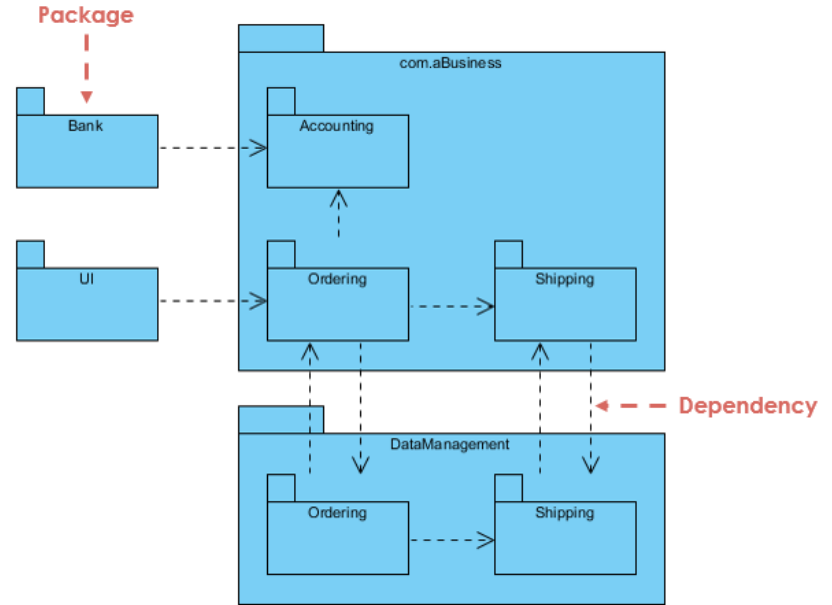
Memory Module (MM)



# Development View

## Package Diagram

Package Diagram can be used to simplify complex class diagrams, it can group classes into packages.



# Deployment or Physical View

The physical view shows the mapping of software onto hardware.

**Shows:**

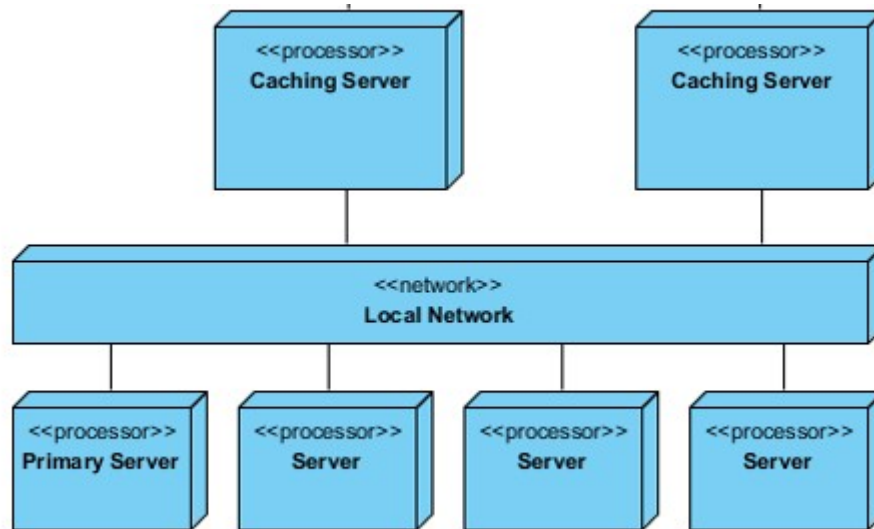
- Servers
- Containers
- Nodes

## Physical View

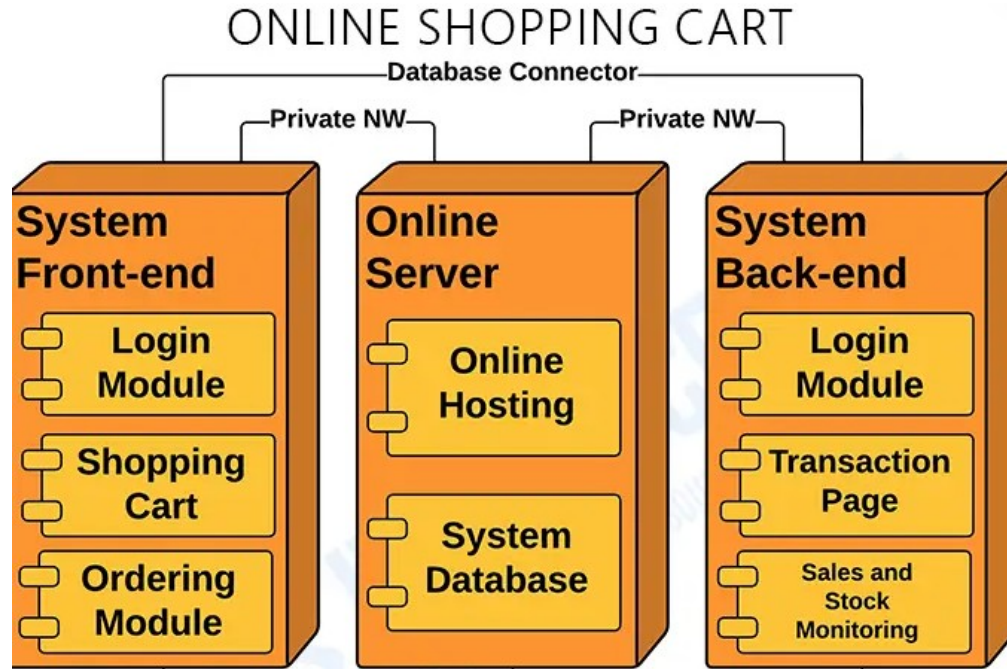
- Deployment Diagram

# Deployment or Physical View

## Deployment Diagram



# Deployment or Physical View



# Logical View

The logical view is based on application domain entities necessary to implement the functional requirements.

The logical view specifies system decomposition into conceptual entities (such as objects) and connections between them (such as associations).

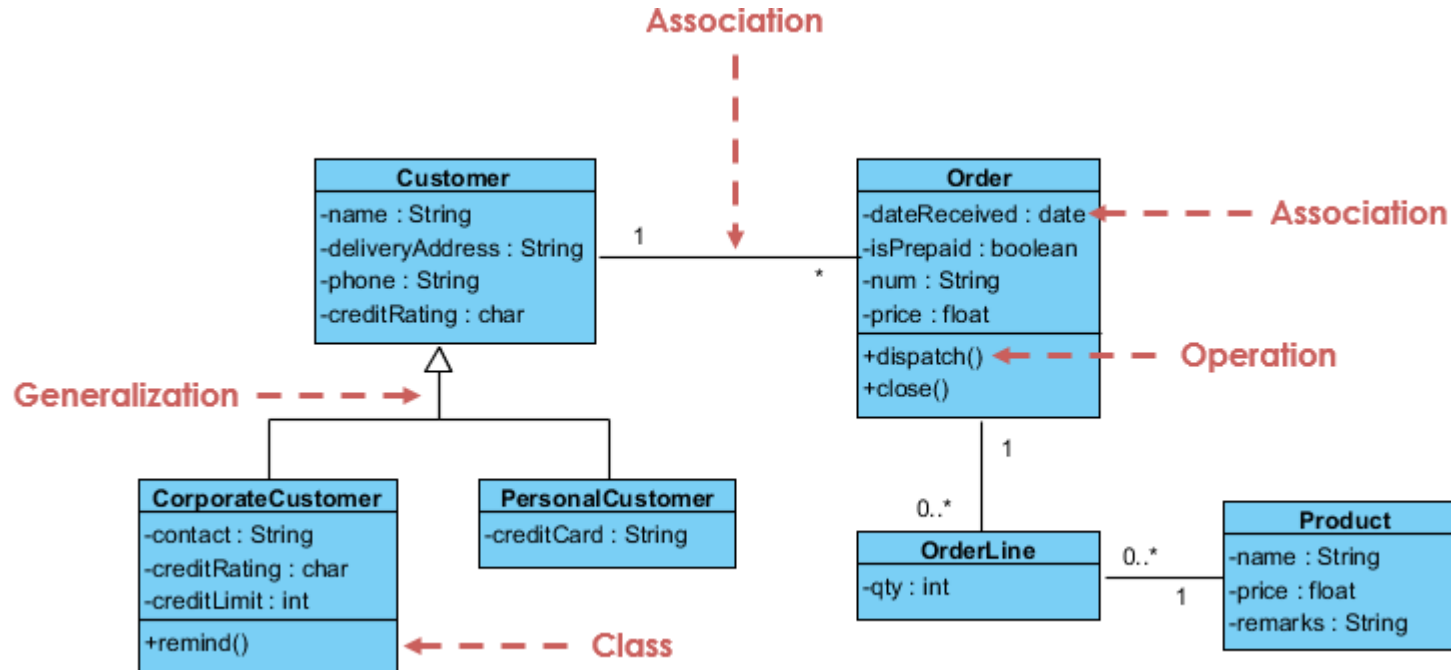
## Logical View

- Class diagram
- State diagram
- Sequence diagram
- Communication



# Logical View

## Class Diagram:



# Software Architectural Styles vs Pattern

| Style                                   | Pattern                                |
|-----------------------------------------|----------------------------------------|
| High-level way of structuring a system. | More concrete / Implementation Details |
| Structure Rules                         | Problem Solving                        |
| Conceptual                              | Practical                              |
| Abstract                                | Detail                                 |

# Software Architectural Styles

(high-level way of structuring a system)

# Categories of Architectural Styles

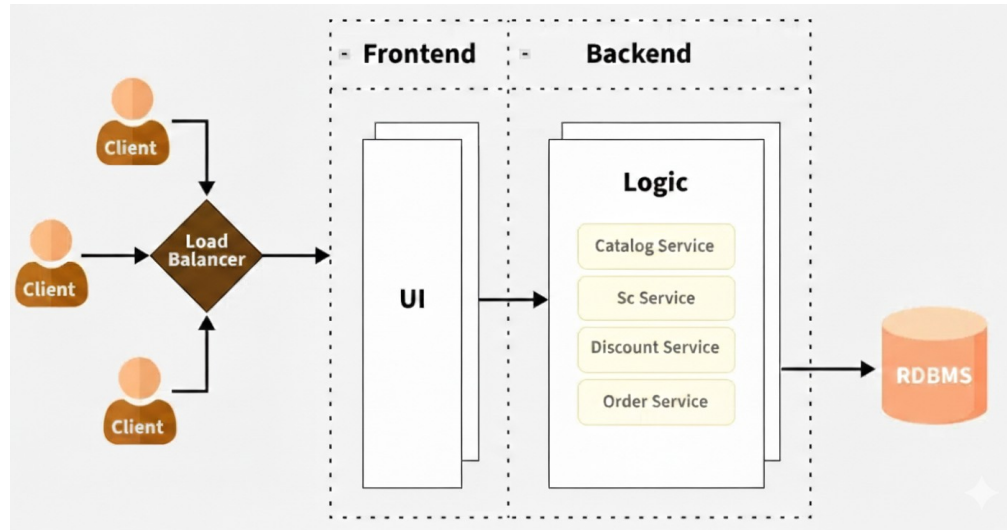
- **Layered Software Architecture**
- **Data Centered Software Architecture**
  - Black board
  - Shared Repository
- **Data Flow Software Architecture**
  - Pipe and Filter
  - Batch Sequential
- **Distributed Software Architecture**
  - Client Server
  - REST
  - SOA
  - Microservices
- **Event Based Software Architecture**

# Layered Software Architecture

# Layered Software Architecture

**Layered architecture (also called n-tier architecture)** is one of the most fundamental and widely used software architectural styles.

It organizes a system into layers, where each layer has a specific responsibility and communicates only with adjacent layers.



# Advantages (Layered Software Architecture)

## 1. Separation of Concerns

Each layer focuses on a single responsibility:

This makes the system conceptually clean and easy to understand.

## 2. Maintainability

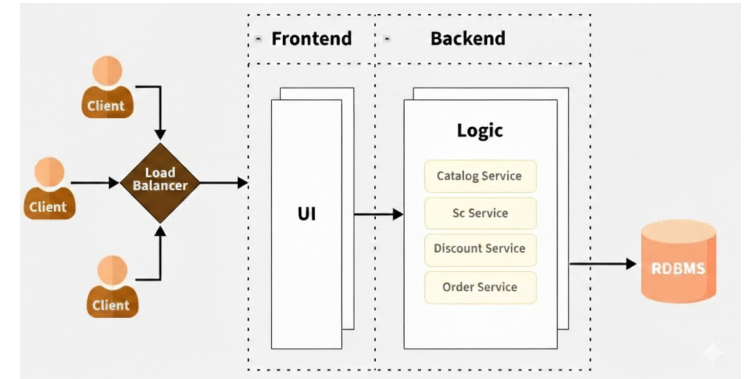
Changes in one layer usually do not affect others.

### Example:

Change database from MySQL to PostgreSQL

Frontend and backend remain unchanged

This reduces maintenance cost.



# Advantages (Layered Software Architecture)

## 3. Team Scalability

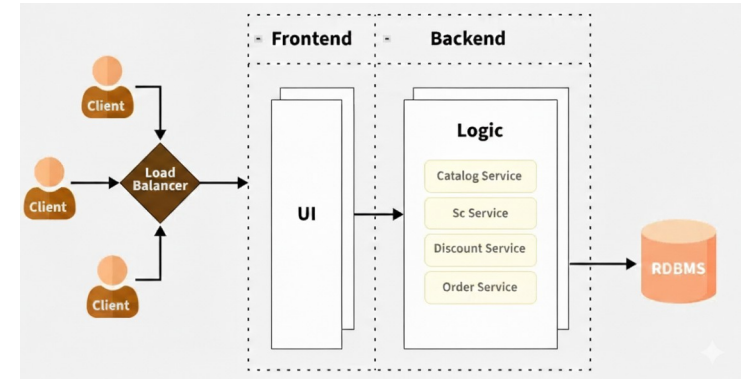
Different teams can work on different layers:

Frontend team

Backend team

Database team

Improves parallel development





# Data Centered or Shared Data Software Architecture

# Shared Data Software Architecture

The software system is decomposed into two major partitions.

- 1) Data Store
- 2) Software components or agents

## **Types:**

Repository Style

Blackboard Style

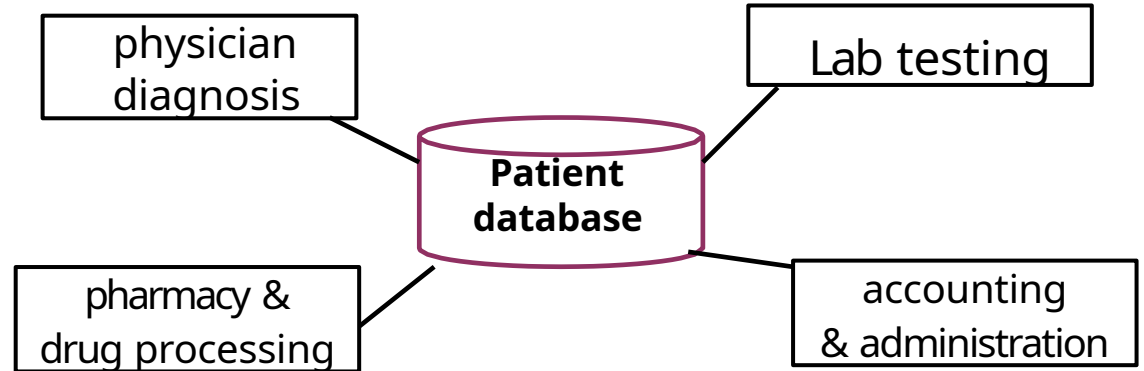
# Repository Software Architecture

The repository architecture style is a data-centered architecture that supports user interaction for data processing.

The software component agents of the data store control the computation and flow of logic of the system.

All data in a system is managed in a central repository that is accessible to all system components.

Components do not interact directly, only through the repository.



# Disadvantages (Repository Architecture)

## 1. Single Point of Failure

If repository fails → whole system fails.

## 2. Performance Bottleneck

High load on central database.

## 3. Tight Data Coupling

All components depend on same schema.

Schema change = break everything.

## 4. Security Risk

One breach exposes everything.

# Blackboard Software Architecture

The Blackboard Architecture is an advanced form of repository (data-centered) architecture used to solve complex, structured problems by allowing multiple independent components (called knowledge sources) to cooperate through a shared central data structure (the blackboard).

## It is used in:

Artificial Intelligence (AI)

- Expert systems
- Pattern recognition
- Robotics
- Computer Vision

## Components:

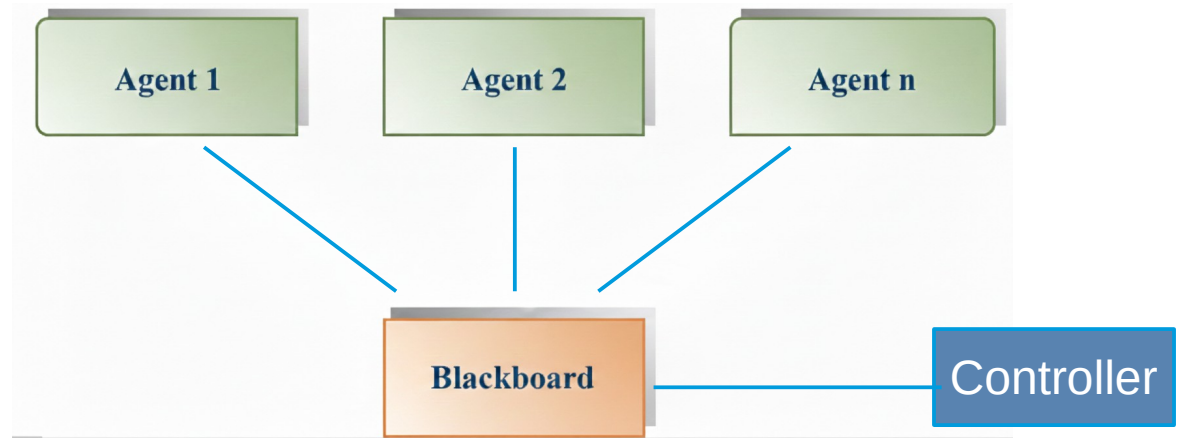
- Blackboard (shared knowledge base)
- Knowledge Sources (experts)
- Control Mechanism (orchestrator)

# Blackboard Software Architecture

## Blackboard:

The blackboard is a global data store that holds:

- Problem description
- Intermediate results
- Partial solutions
- Final solution



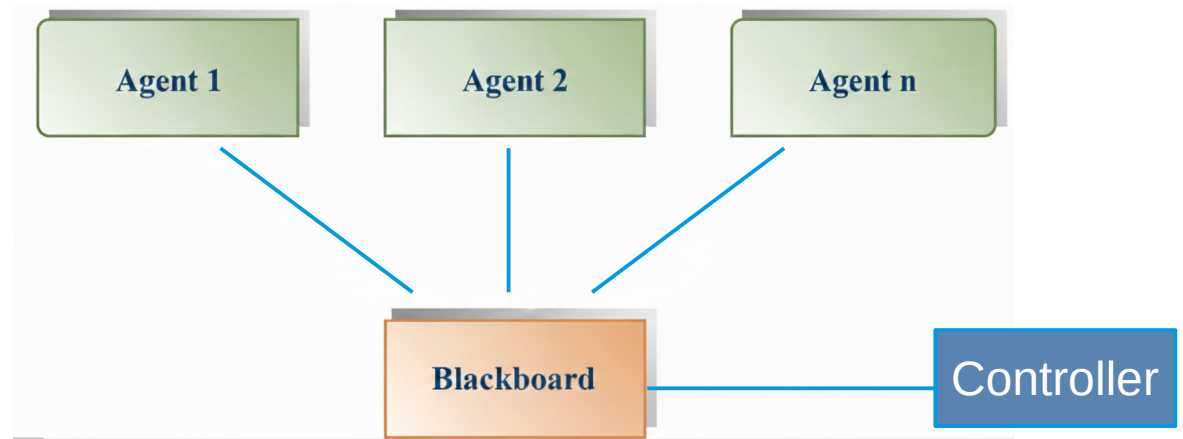
# Blackboard Software Architecture

## Knowledge Source:

Independent problem solvers.

### Each Knowledge Source:

- Specializes in one task
- Has no knowledge of other KS
- Activates when relevant data appears

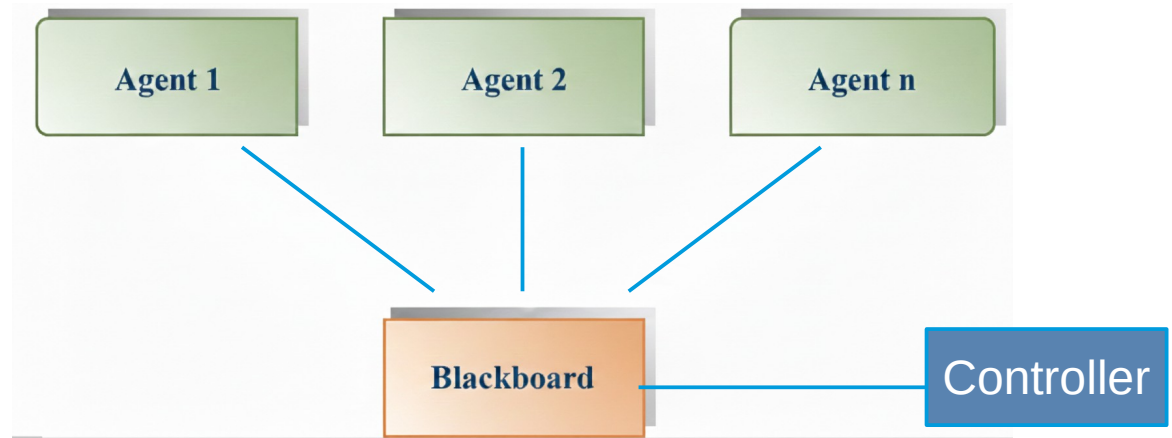


# Blackboard Software Architecture

## Controller:

- Continuously monitors the blackboard,
- Decides which knowledge source (KS) should execute next,
- Resolves conflicts between competing KS,
- Applies global problem-solving strategies.

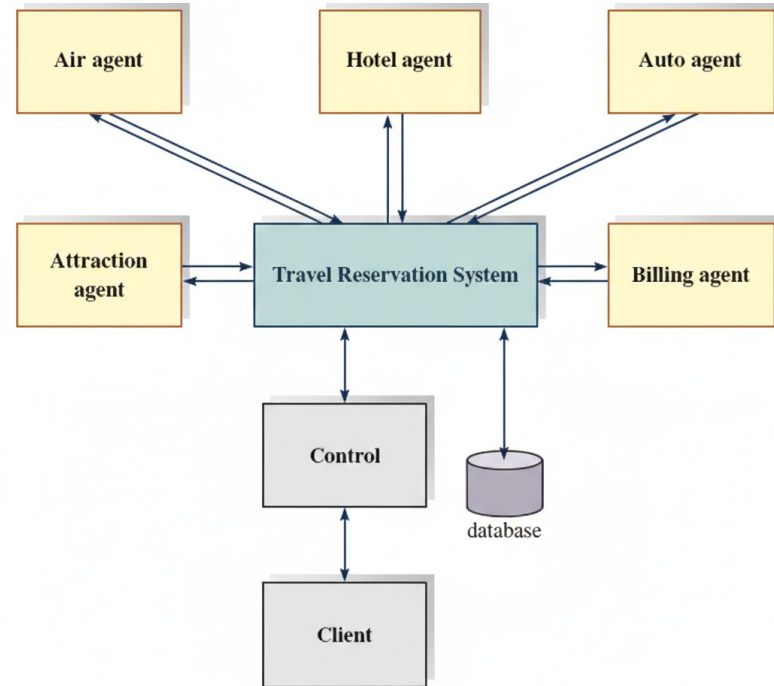
It is what makes blackboard a thinking system instead of a storage system.





# Blackboard Software Architecture

## Travel Consulting System



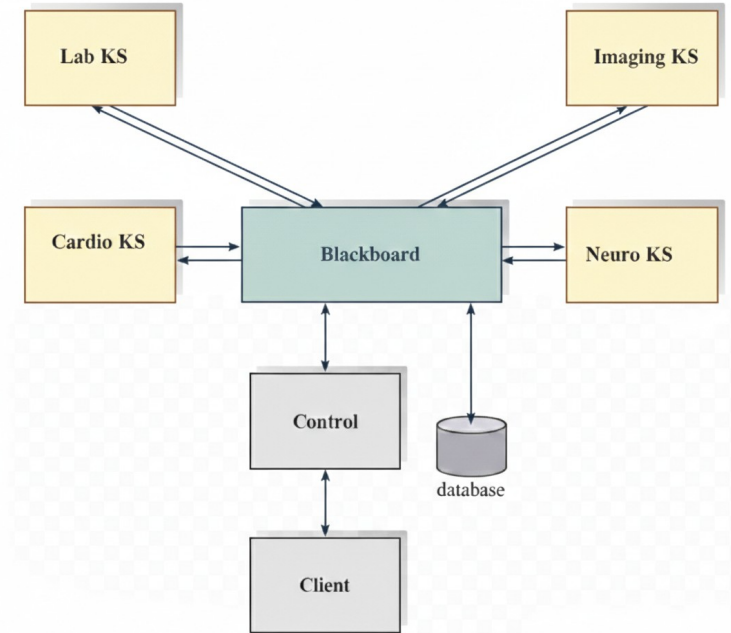
# Blackboard Software Architecture

## Medical Expert System

A hospital wants an AI-based diagnosis system that:

- Accepts patient symptoms
- Uses multiple medical specialties
- Produces a ranked diagnosis list

There is no fixed algorithm for diagnosis. Different experts contribute knowledge.



# Blackboard Software Architecture

## Working of Medical Expert System:

### Component 1: Blackboard (Shared Knowledge Base)

The blackboard stores all patient knowledge in multiple levels.

**Level 1:** Raw symptoms (fever, pain)

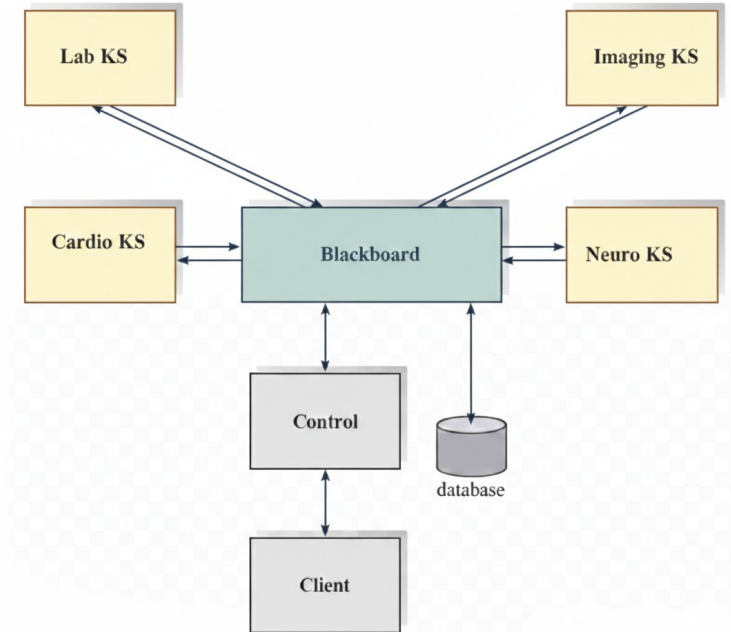
**Level 2:** Clinical signs

**Level 3:** Lab interpretations

**Level 4:** Disease hypotheses

**Level 5:** Final diagnosis

Patient:  
fever = 39°C  
chest pain = true  
ECG = abnormal  
WBC = high  
This is visible to all knowledge sources.



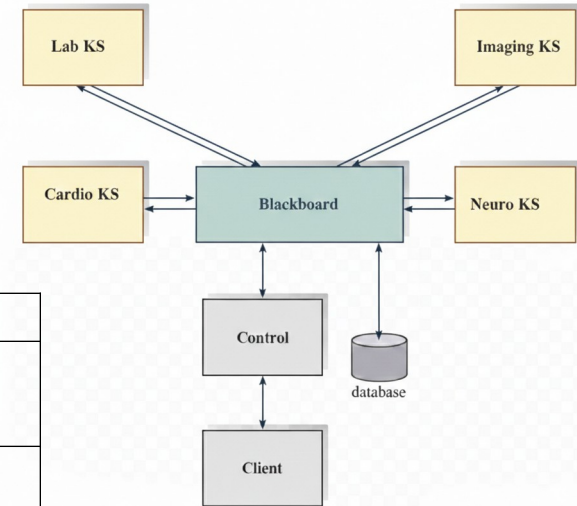
# Blackboard Software Architecture

## Component 2: Knowledge Source

Each KS is an independent medical expert.

They do NOT know about each other.

| Knowledge Source              | Trigger Condition              | Action                                                                |
|-------------------------------|--------------------------------|-----------------------------------------------------------------------|
| Cardiology Knowledge Source   | If chest pain AND abnormal ECG | Adds hypothesis:<br>Possible Myocardial Infarction<br>Confidence: 0.7 |
| Neuro Knowledge Source        | If headache AND vision loss    | Action xyz                                                            |
| LAB Analysis Knowledge Source | If WBC > threshold             | Possible Infection<br>Inflammatory Response Detected                  |
| Imaging Knowledge Source      | When X-ray or MRI available    | Lung opacity detected<br>Possible pneumonia                           |



# Blackboard Software Architecture

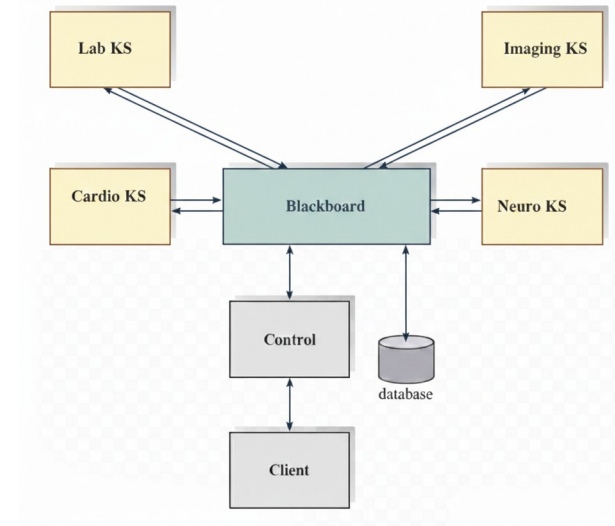
## Component 3: Control Mechanism

- Monitors blackboard
- Chooses which KS runs next
- Resolves conflicts

## Final Blackboard State

### Diagnosis:

1. Severe Pneumonia (0.8)
2. Cardiac stress (0.6)
3. Infection (0.9)



# Data Flow Software Architecture

# Data Flow Software Architecture

Batch Sequential Style

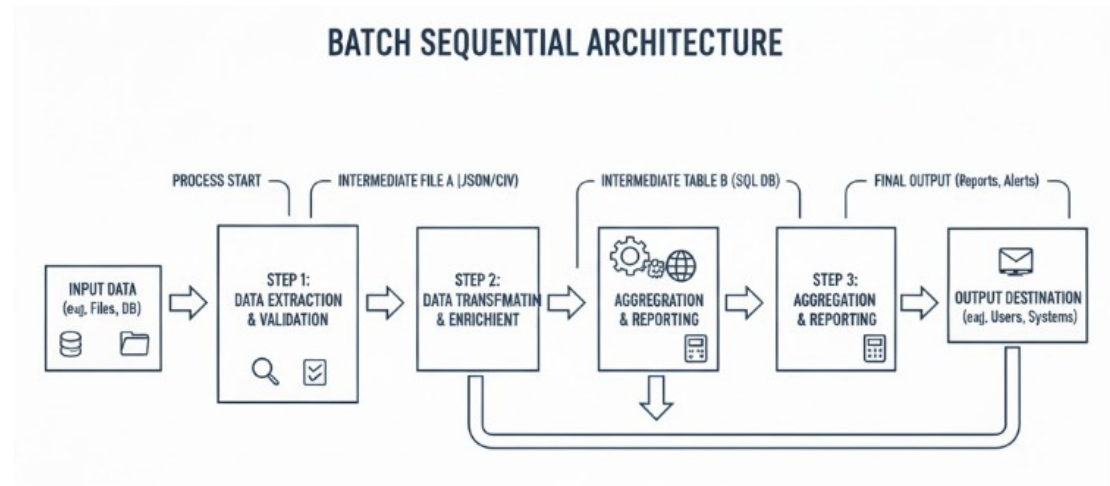
Pipe and Filter Style

# Batch Sequential Software Architecture

Batch Processing Architecture is a software architectural style where large volumes of data are collected, stored, and processed together as a group (batch) at scheduled times, instead of being processed immediately.

It is one of the oldest and most reliable architectures, still heavily used in:

- Banking systems (for some operations)
- Payroll systems
- Billing systems





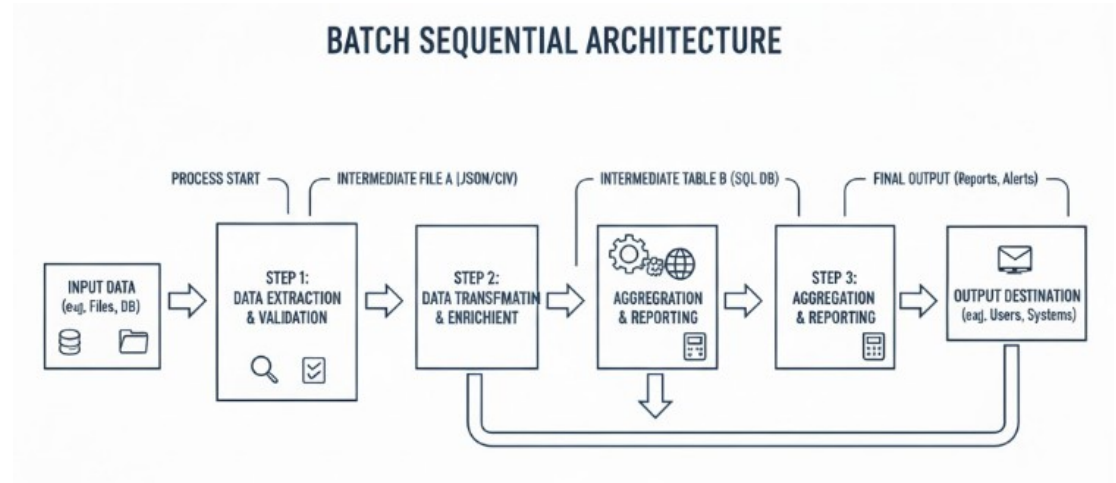
# Batch Sequential Software Architecture

## Modern Implementations

Big Data

### When to Use?

- Data volume is massive
- Real-time not required
- Historical analysis



# Batch Sequential Software Architecture

## Disadvantages

- High Latency
- Not Interactive
- Resource Spikes
- Not suitable for Real-Time Systems

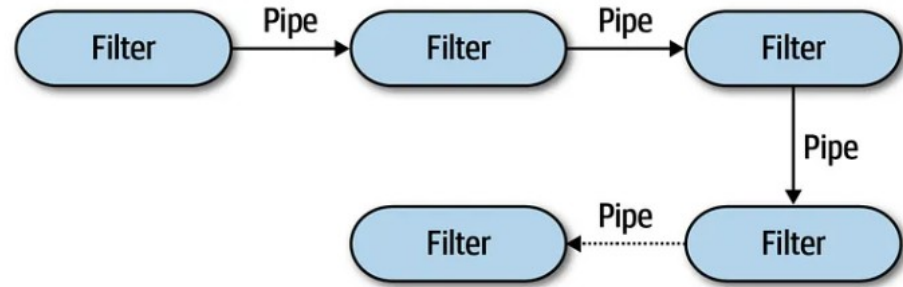
# Pipe and Filter Software Architecture

## Pipe and Filter:

Pipe and Filter architecture is a data-flow architectural style in which data is processed by a sequence of independent components (filters) connected by communication channels (pipes).

### Each filter:

- Takes input
- Processes it
- Produces output



# Pipe and Filter Software Architecture

## Advantages:

### Parallelism

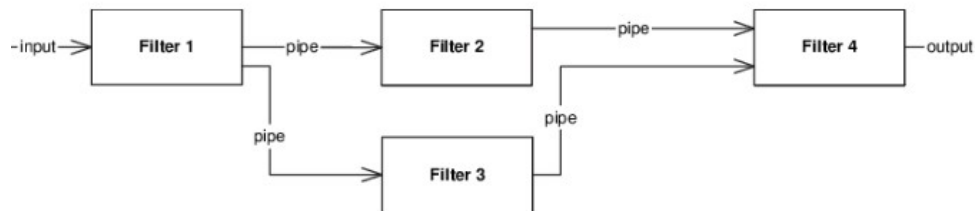
Different filters can run concurrently.

### Maintainability

Change one filter → rest unaffected.

### Scalability

Add more filters or replicate them.



# Pipe and Filter Software Architecture

Disadvantages:

**Not Good for Interactive Systems**

**Error Handling is Hard**

- If one filter fails, Entire pipeline breaks

# Pipe and Filter Software Architecture

## Modern Implementations:

### **DevOps**

CI/CD pipelines

### **Machine Learning**

Data → Cleaning → Feature Extraction → Model → Evaluation

# Difference between Pipe and Filter and Batch Sequential

| Batch Sequential | Pipe & Filter  |
|------------------|----------------|
| Whole dataset    | Stream         |
| Sequential       | Parallel       |
| Time-triggered   | Data-triggered |

# Distributed Software Architecture



# Distributed Software Architecture

Client Server

REST

SOA

Microservices

# Client Server

Client-Server Architecture is an architectural style in which software is structured around two main roles:

Client → Requests services

Server → Provides services

The client sends a request over a network, and the server processes the request and returns a response.

# Client Server

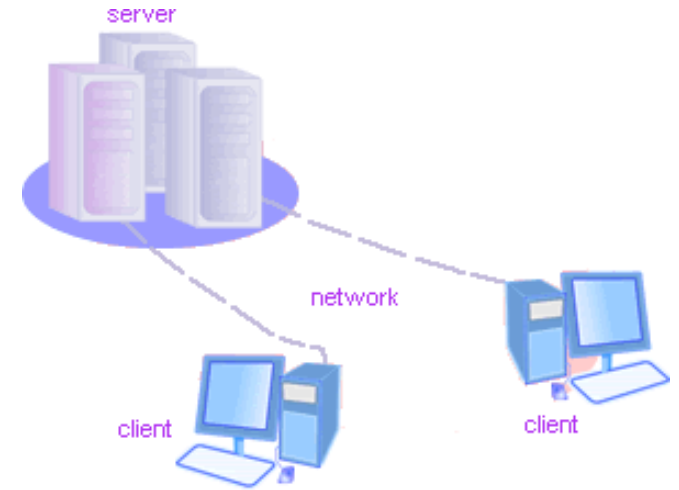
Suitable for applications that involve distributed data and processing across a range of components.

## Components:

**Servers:** Stand-alone components that provide specific services such as printing, data management, etc.

**Clients:** Components that call on the services provided by servers.

**Connector:** The network, which allows clients to access remote servers.



# Client Server – Example of Clients

Web browsers

Mobile apps

Desktop apps

ATM machines

# Client Server – Types of Server

## **File Servers:**

Useful for sharing files across a network.

The client passes requests for files over the network to the file server.

## **Database Servers:**

Client passes SQL requests as messages to the DB server; results are returned over the network to the client.

Query processing done by the server.

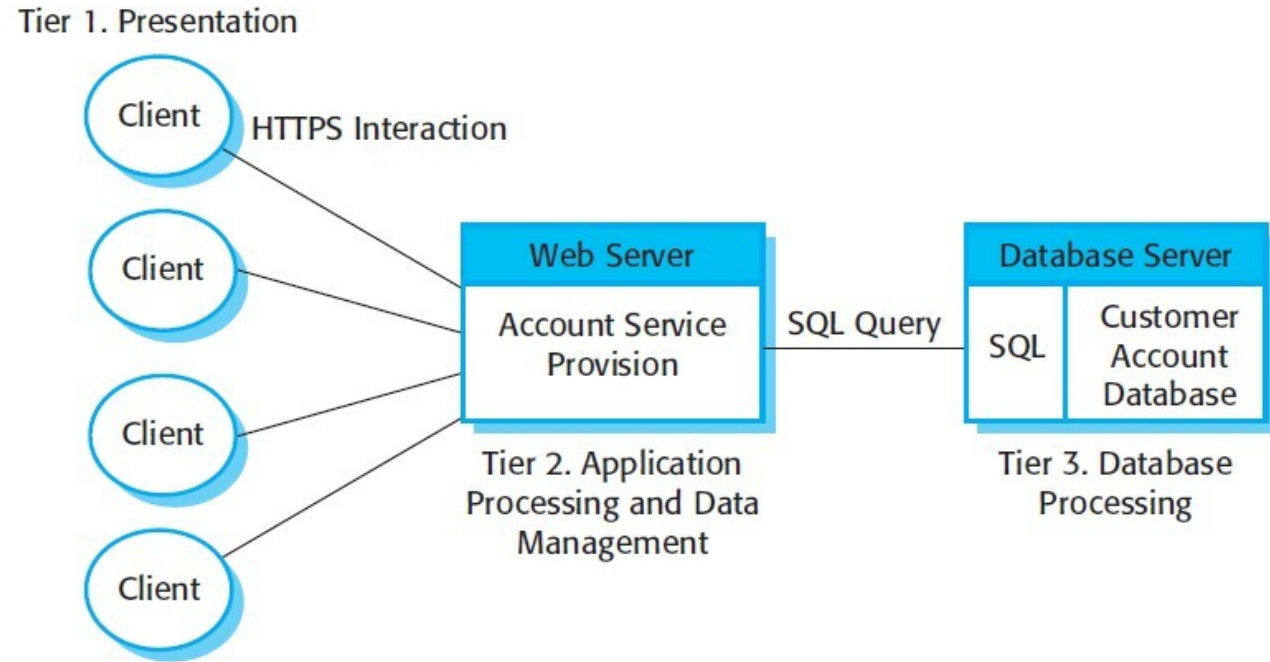
No need for large data transfers.

# Client Server – Network

## **Uses protocols like:**

- HTTP / HTTPS
- FTP
- WebSocket

# Multi-Tier Client Server



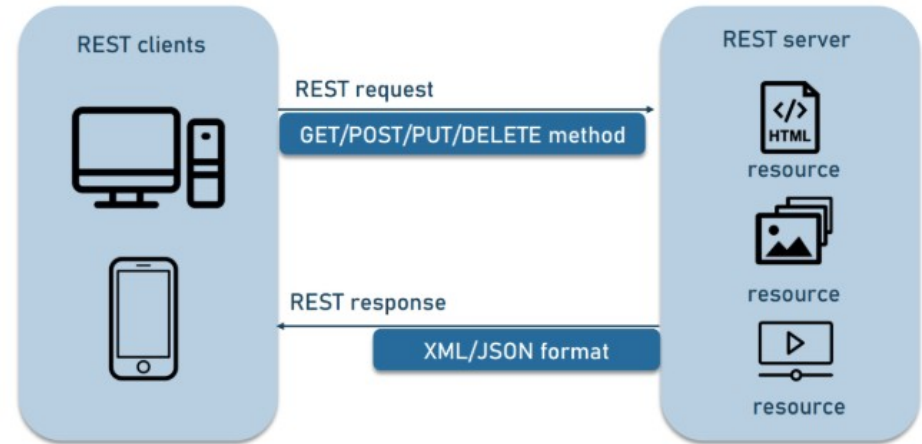
**Figure 18.10** Three-tier architecture for an Internet banking system

# REST

REST (Representational State Transfer) is an architectural style for designing networked applications.

Roy Fielding introduced and developed REST in his doctoral thesis in 2000.

RESTful services are widely used in modern Web architectures.





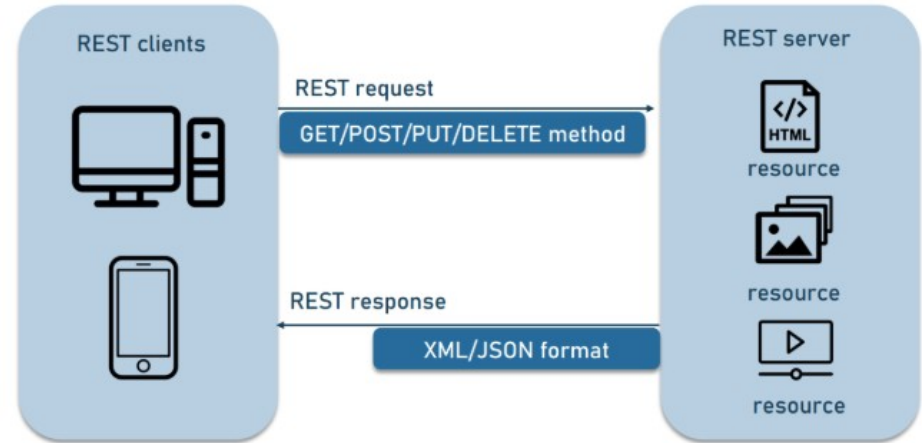
# REST

## HTTP REST

### Request/Response Model

REST allows applications to communicate with each other by carrying JSON data between the client and server.

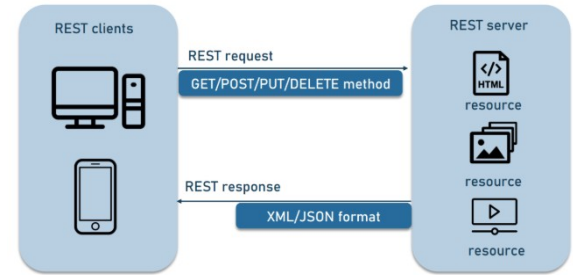
REST help us in building performant and scalable applications.



# REST

## Key Characteristics of HTTP REST:

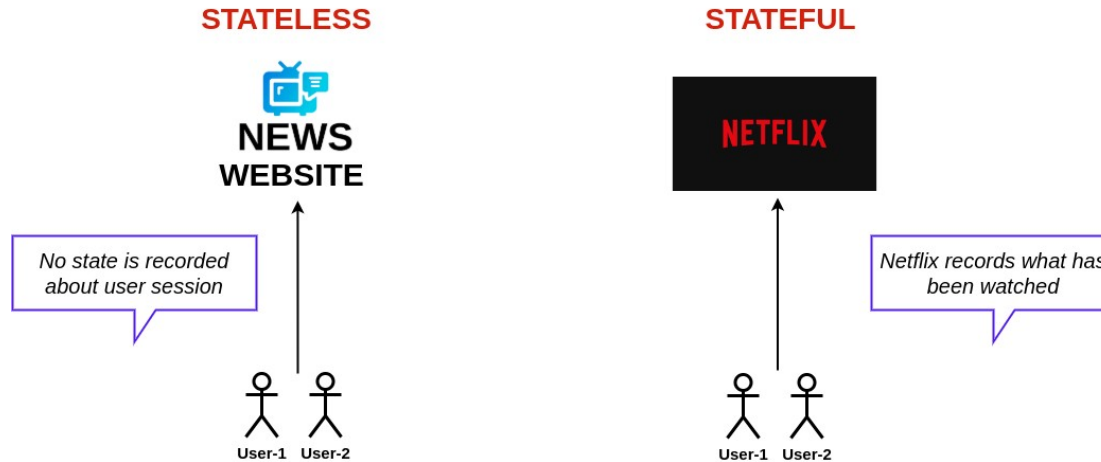
1. **Stateless** (Each HTTP request from a client must contain all the information the server needs to fulfill that request. The server does not store any state about the client session.)
2. **Uniform Interface** (REST uses a standardized way to interact with resources, usually by using URIs (Uniform Resource Identifiers))
3. **Cacheable** (Responses should be defined as cacheable or non-cacheable to improve efficiency.)
4. **Client-Server** (REST divides the system into clients and servers, with the client making requests and the server responding.)
5. **Layered System** (Architecture can be composed of layers that work together without the client being aware of the different layers)



# REST

## Stateless Vs Stateful:

**Stateless** (Each HTTP request from a client must contain all the information the server needs to fulfill that request. The server does not store any state about the client session.)



# REST

## Uniform Resource Identifiers

*REST API's are designed around "resources".*

*Every Resource has an identifier.*

*The best practice is the resource URIs should be based on nouns not verbs.*

*REST API Url: <https://e-shop.com/products>*

| HTTP Methods |                       |
|--------------|-----------------------|
| GET          | Retreives Information |
| POST         | Submit data to server |
| PUT          | Update data           |
| DELETE       | Delete data           |

*A resource can be anything that the API interacts with, such as:*

**Entities:** Users, products, orders, etc.

**Data:** Files, images, videos, etc.

| Status Codes     |                         |
|------------------|-------------------------|
| 200 ok           | Successful Request      |
| 201 created      | Resource Created        |
| 401 unauthorized | Authentication Required |
| 404 not found    | Resource not found      |

# REST

## Examples:

# Server stores session

```
session["user_id"] = 101
```

# Server checks session memory

```
if session["user_id"]:  
    allow_access()
```

Server remembers the user. This is stateful.

## Stateless (RESTful way)

Client sends authentication token in every request:

```
GET /orders  
Authorization: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

## Server:

- Verifies token
- Processes request
- Does NOT store anything about client
- Next request must again send token.

# Stateful → Effect on Scalability

## **Load Balancer Issue:**

Suppose you have:

- Server A
- Server B
- Load Balancer

User logs in → session stored in Server A

## **Next request:**

If Load Balancer sends request to Server B

Server B does NOT have session

User gets logged out.

# Stateful → Effect on Scalability

## Shared Storage Session Issue

To fix above issue, you store sessions in:

Database

Now architecture becomes:

User → Load Balancer → App Servers → Shared Session Store

New problems:

- Extra network calls
- Slower performance
- More infrastructure cost
- More complexity
- Single point of failure (if session store crashes)

# Stateful → Effect on Scalability

## **Memory Limitation Issue:**

If session stored in RAM:

1 million users

Each session = 5KB

Total = 5GB RAM needed just for sessions



# Stateful → Effect on Reliability

## **Server Crash**

If session stored in memory:

### **Server crashes then**

- All sessions lost
- All users logged out

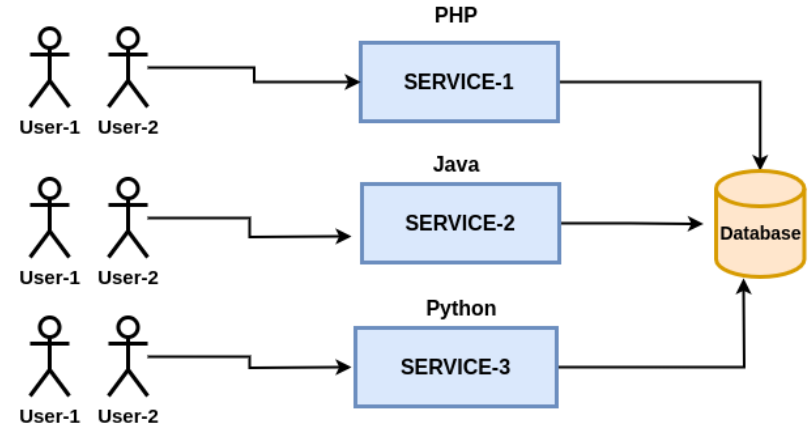
System becomes unreliable.

Users cannot authenticate.

# Service Oriented Architecture

A service-oriented architecture (SOA) is an architectural style in which application components provide services to other components via a communications protocol, typically over a network.

The principles of service-orientation are independent of any product, vendor or technology.



# Service Oriented Architecture

A service is a self-contained unit of software that performs a specific task or set of tasks.

It can be *accessed remotely*, often over a network, and it provides a **well-defined interface** to its consumers.

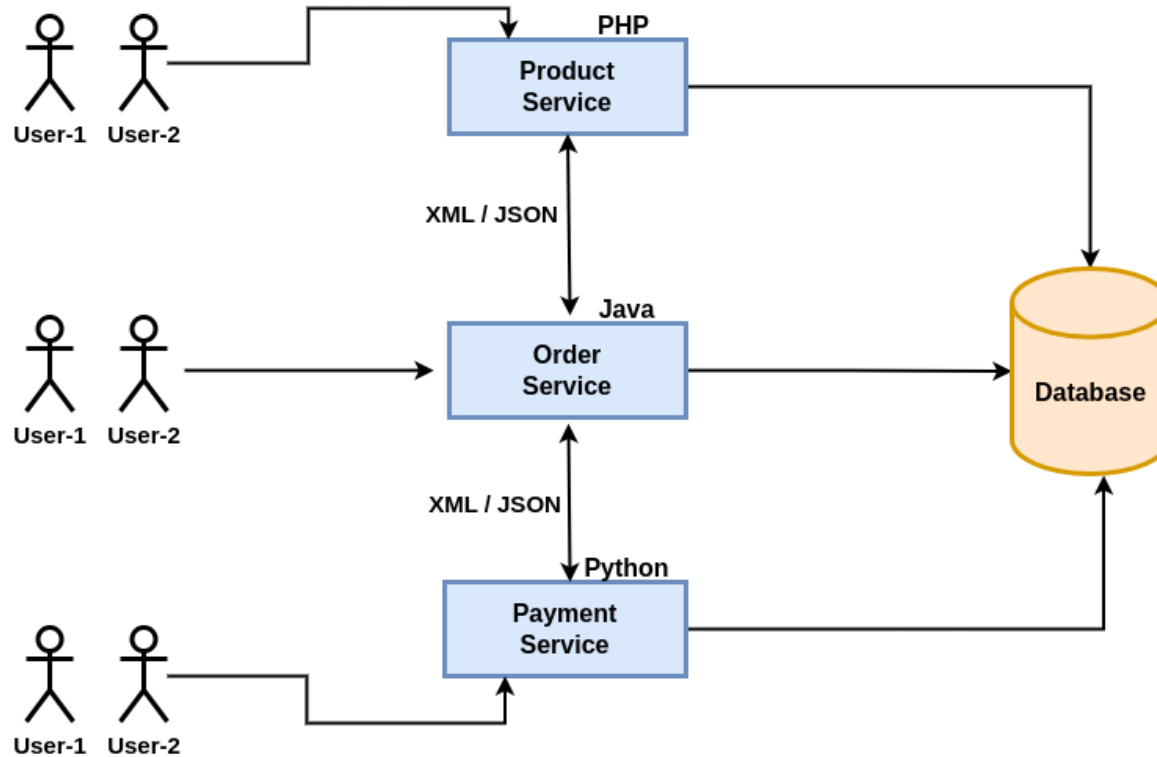
Services are designed to be independent, modular, and reusable across different applications and systems.

## **Well Defined Interface:**

SOAP (Lightweight XML based protocol) for data exchange between services.

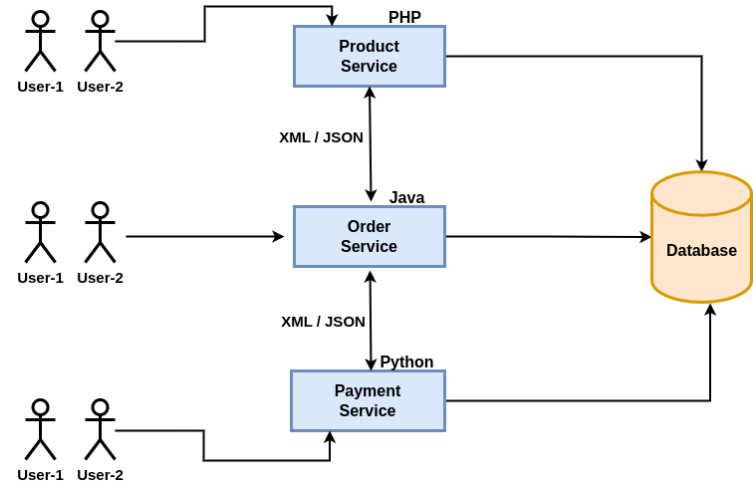
JSON (JavaScript Object Notation)

# Service Oriented Architecture



# Limitations of SOA

Services are independent of any product, vendor or technology but still all services are dependent on one back end service of database as shown in the diagram.



# Microservices

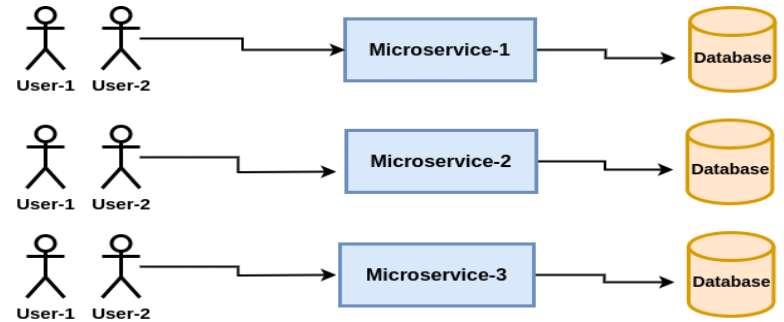
Microservices are **small, independent**, and **loosely coupled** services that can work together.

Each service is a **separate codebase**, which can be managed by a small development team.

Each Microservice can be developed with a separate **technology stack**.

Microservices communicate with each other by using **well-defined APIs**.

Each Microservices has its own **database**.

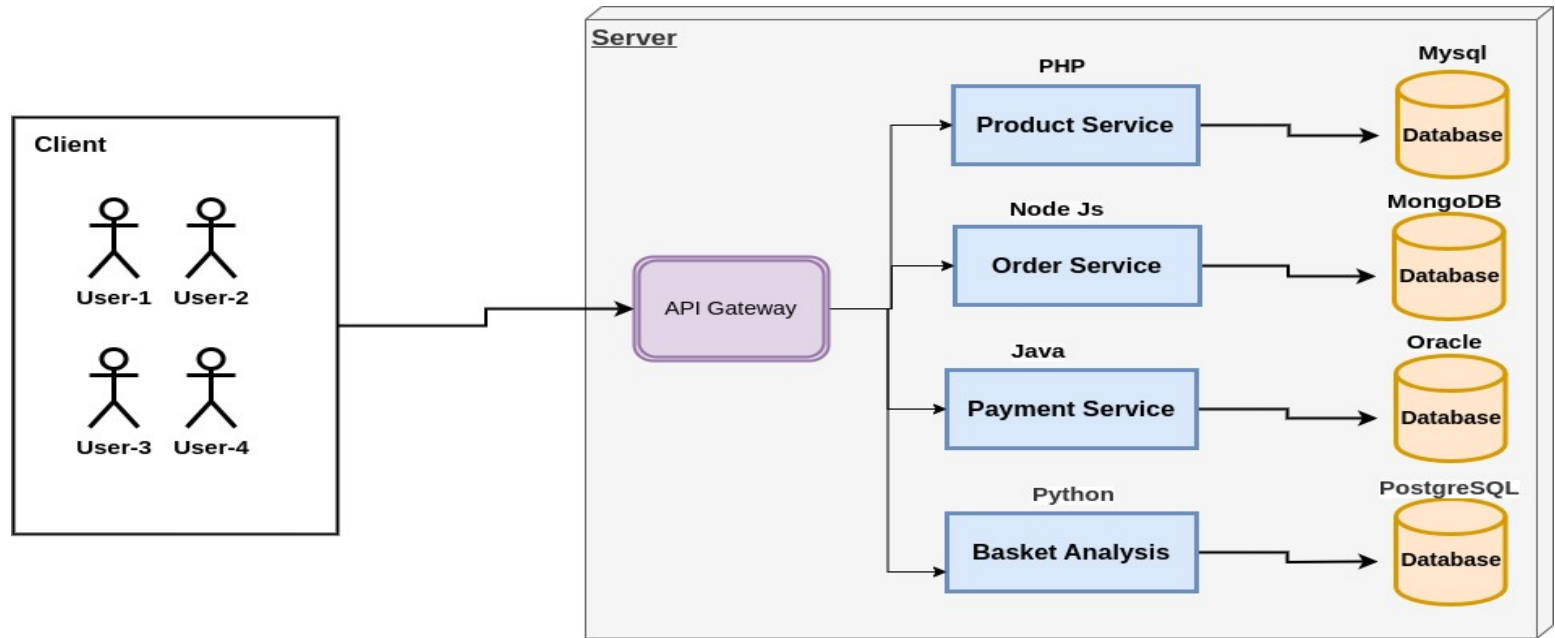


# Microservices

Separate Codebase

Separate Technology Stack

Separate Database



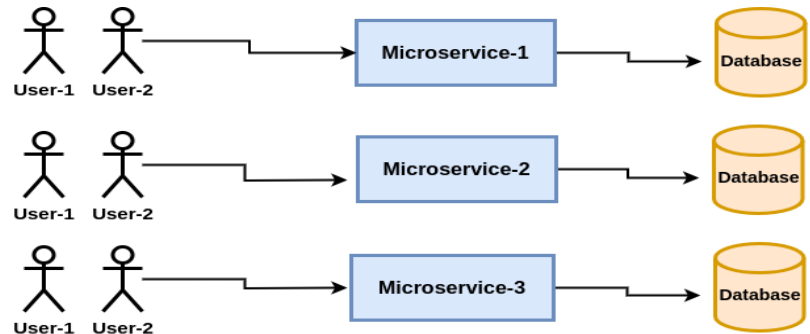
# Benefits of Microservices

## Agility

Microservices architectures make applications easier to scale and faster to develop, enabling innovation and accelerating time-to-market for new features.

## Flexibility and Scalability

Microservices can be scaled independently, so you scale out sub-services that require less resources, without scaling out the entire application.





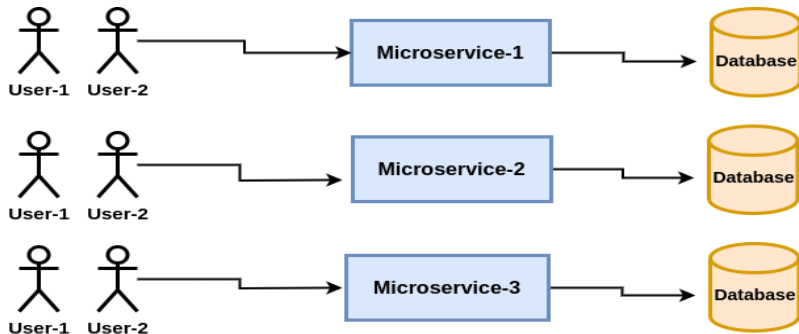
# Benefits of Microservices

## Small, focused teams

Microservices should be small enough that a single feature team can build, test, and deploy it.

## Small and separated code base

Microservices are not sharing code or data stores with other services, it minimizes dependencies, and that makes easier to adding new features.



# Benefits of Microservices

## Easy Deployment

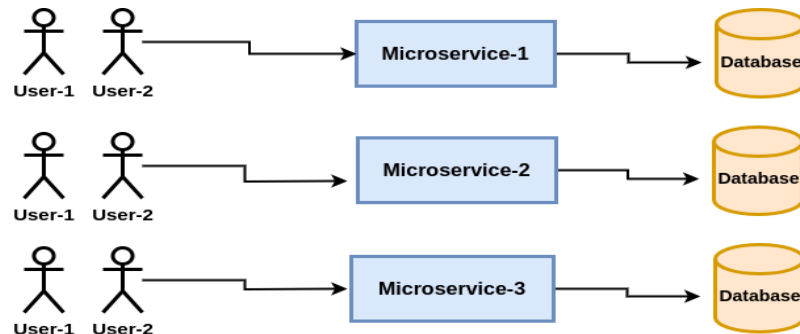
Microservices enable continuous integration and continuous delivery, making it easy to try out new ideas and to roll back if something doesn't work.

## Technology agnostic, Right tool for the job

Small teams can pick the technology that best fits their microservice and using a mix of technology stacks on their services

## Data isolation

Databases are separated with each other according to microservices design. Easier to perform schema updates, because only a single database is affected.



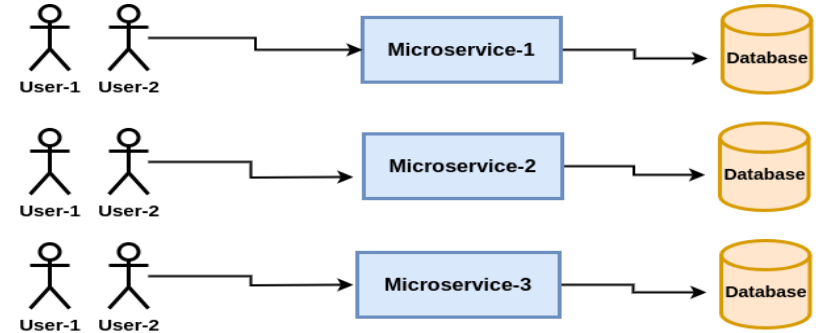
# Challenges of Microservices

## Complexity

Each service is simpler, but the entire system is more complex. Deployments and Communications can be complicated for hundreds of microservices.

## Deployment

Deployments are challenging. Require to invest in quite a lot of devops automation processes and tools. The complexity of microservices becomes overwhelming for human deployment.



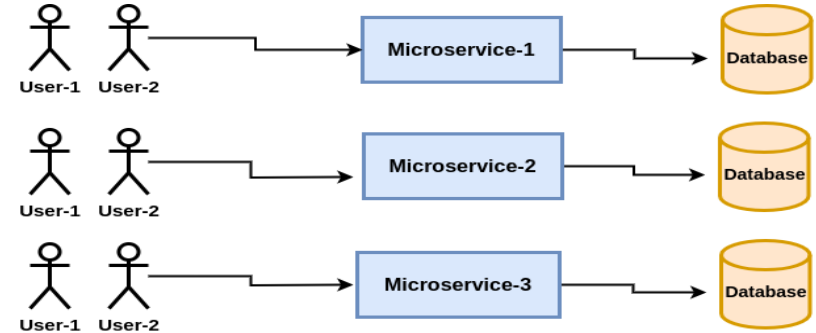
# Challenges of Microservices

## Network problems and latency

Network latency is the time it takes for data to travel from one point to another over the network.

In a microservices architecture, different services communicate with each other over the network. Each time one service needs data or functionality from another service, it has to send a request over the network and wait for a response.

**Impact:** As the number of services grows, the cumulative latency from these network calls can affect the overall performance of the application.



# Challenges of Microservices

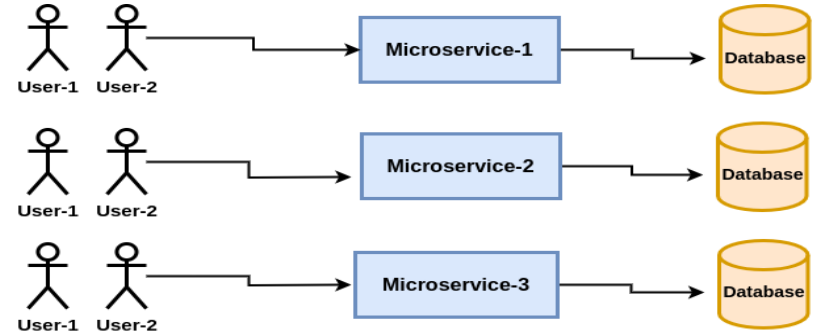
## Fault Tolerance

Fault tolerance refers to the system's ability to continue operating properly even when part of the system fails.

Since microservices are distributed and often run on different servers or containers, any individual service might fail due to various reasons, such as network issues, server crashes, or bugs.

To ensure that the whole system remains operational, microservices architectures need the underlying mechanisms for fault tolerance.

- Retries:
- Circuit Breakers:
- Fallbacks:
- Load Balancing:



# Event Driven Architecture

Event-Driven Architecture (EDA) is a software architecture style where components communicate through events.

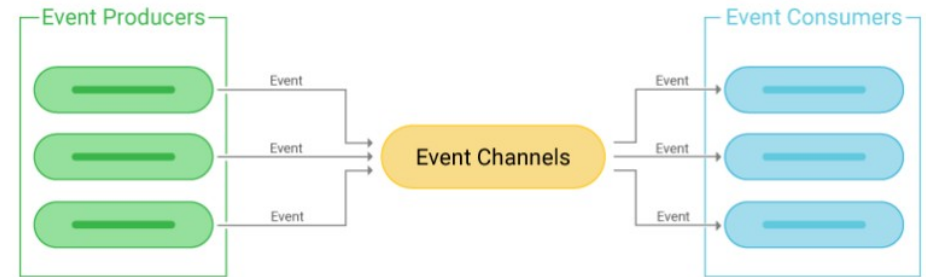
Instead of services calling each other directly, they produce events when something happens, and other services react to those events.

An event represents a change in state or an important occurrence in a system.

**Example:**

- A user places an order.
- A payment is completed.
- A sensor detects temperature change.

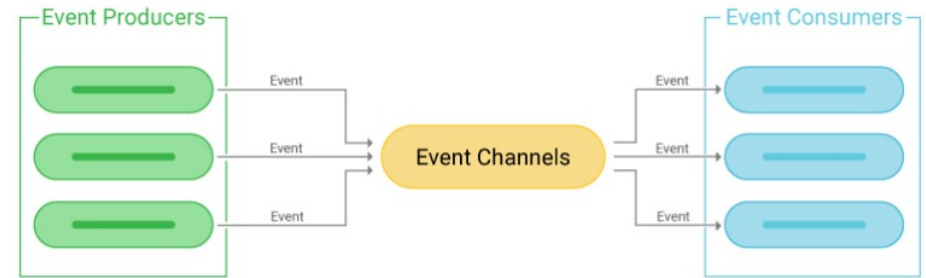
Each of these actions generates an event, which other components can consume and process.



# Event Driven Architecture

## Components:

- Event Producer
- Event Consumer
- Event Broker / Event Bus
- Event



# Event Driven Architecture

## Event Producer:

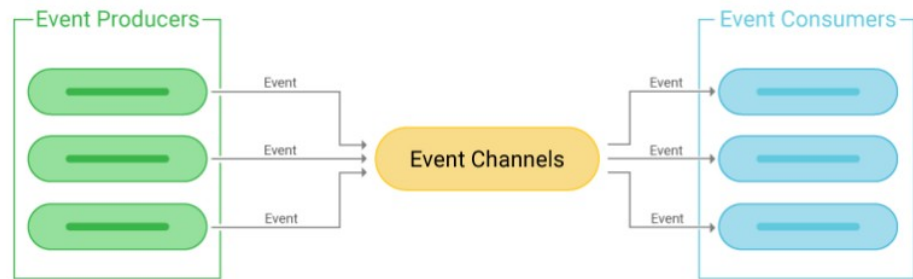
The component that generates an event.

### **Example:**

- Order service generates OrderPlaced event
- Temperature sensor generates TemperatureChanged event

### **Responsibilities:**

- Detect state change
- Create event
- Send event to event broker





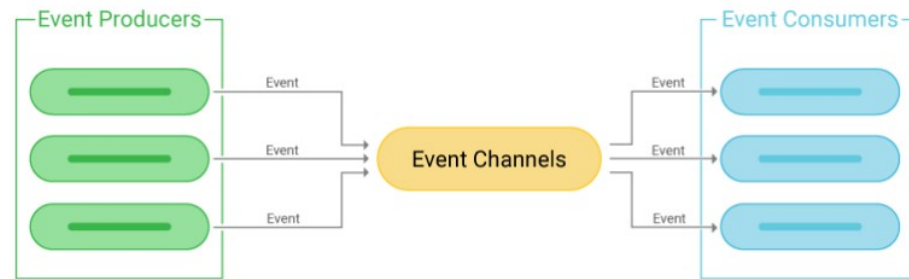
# Event Driven Architecture

## Event Consumer:

The component that receives and processes events.

### Example:

- Payment Service receives OrderPlaced event and processes payment.
- Inventory Service receives OrderPlaced event and updates stock.



One event may have multiple consumers.

# Event Driven Architecture

## Event Broker / Event Bus:

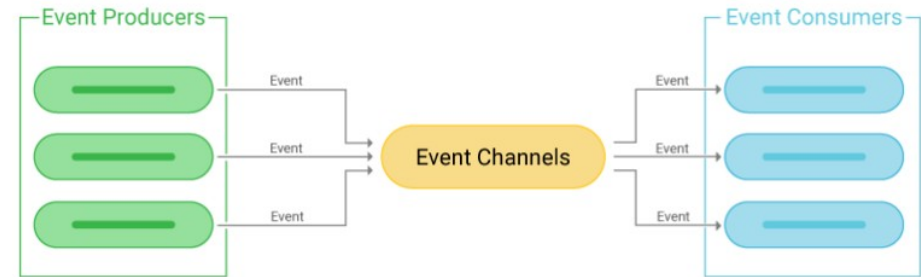
The middle layer that distributes events.

It manages:

- Event routing
- Message queues
- Event delivery

The broker ensures that:

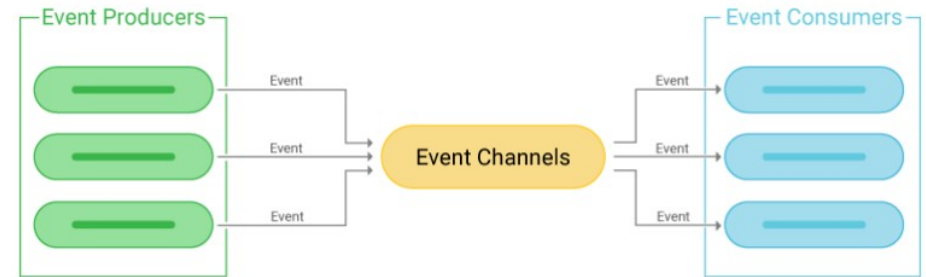
- Events reach the correct consumers
- Messages are not lost



# Event Driven Architecture

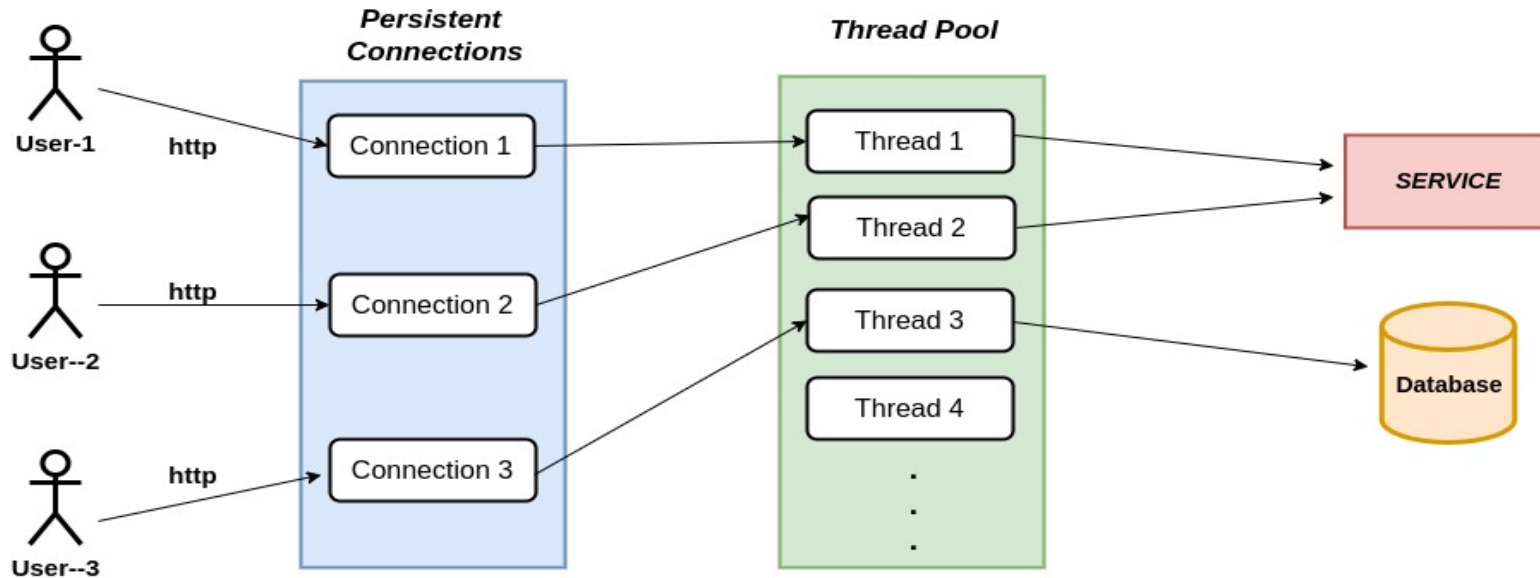
## Event Flow in EDA:

- 1) User places order.
- 2) Order Service creates OrderPlaced event.
- 3) Event is sent to event broker.
- 4) Broker distributes event to subscribers.
- 5) Services process event independently.



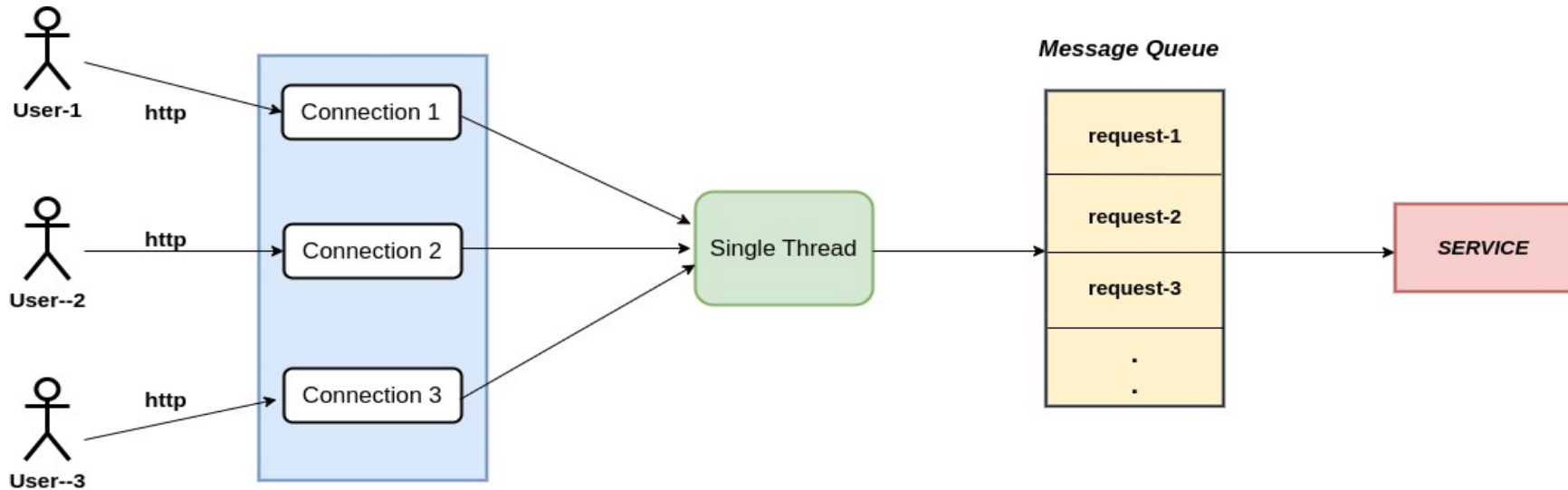
# Synchronous vs Asynchronous

## Synchronous Communication (Request Response Model):



# Synchronous vs Asynchronous

## Asynchronous Communication (Event Driven Model):



# Event Driven Architecture

## Advantages:

|                      |                                                                                                         |
|----------------------|---------------------------------------------------------------------------------------------------------|
| Loose Coupling       | Services do not know about each other.<br>They only know events.                                        |
| High Scalability     | Consumers can scale independently.<br>Example:<br>More consumers can be added to process events faster. |
| Real-Time Processing | EDA supports real-time data streaming.<br>Example:<br>Fraud detection<br>IoT sensor monitoring          |
| Flexibility          | New services can subscribe to existing events without modifying producers.                              |

# When to not use EDA

## When Immediate Response is Required (Synchronous Systems)

EDA is asynchronous, meaning the producer sends an event and does not wait for the response.

However, some systems require immediate feedback.

### **Example**

ATM withdrawal system.

Steps required:

- ATM sends withdrawal request.
- Bank system checks balance.
- Bank deducts amount.
- Bank confirms transaction.
- ATM dispenses cash.

All must happen synchronously and immediately.

### **If EDA is used:**

ATM might dispense cash before balance confirmation.

Events may be delayed.

### **Better approach:**

Request–Response architecture (REST APIs).

Two ATMs send events simultaneously.

Example:

Balance = \$500

Two withdrawals:

ATM1 → Withdraw 400

ATM2 → Withdraw 400

# Software Architecture Patterns



# Categories of Architectural Pattern

- **Model View Controller**
- **Command Query Responsibility Segregation**
- **Clean Architecture**

# Model View Controller

The Model–View–Controller (MVC) Pattern is one of the most widely used architectural patterns in software engineering for structuring applications.

It separates an application into three interconnected components so that development, maintenance, and testing become easier.

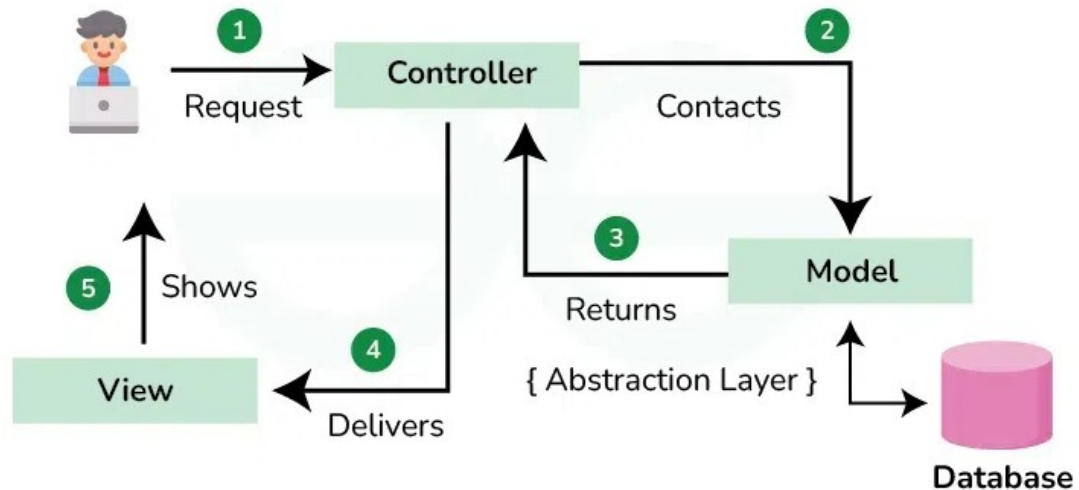
**MVC (Model–View–Controller)** divides an application into three layers:

**Model** → Data and business logic

**View** → User interface (UI)

**Controller** → Handles user input and connects Model and View

The main goal is Separation of Concerns (SoC).



# Model View Controller

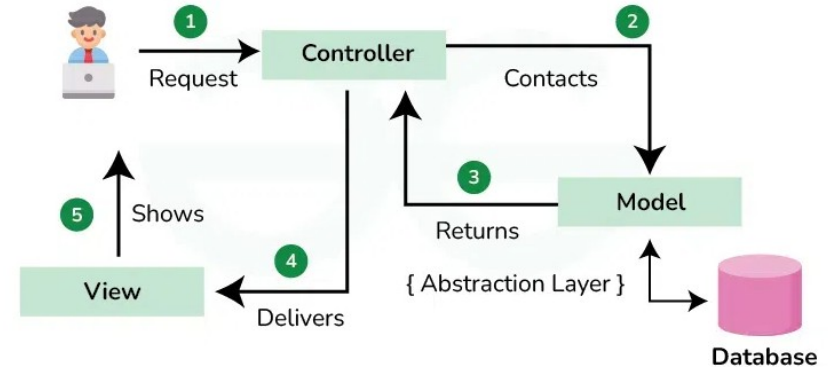
## 1. Model

The Model represents the data and the business logic of the application.

It manages the state of the application and usually communicates with a database.

**Responsibility:** Handling data validation, storage, and retrieval.

**Rule:** The Model does not know anything about how the data is displayed to the user.



# Model View Controller

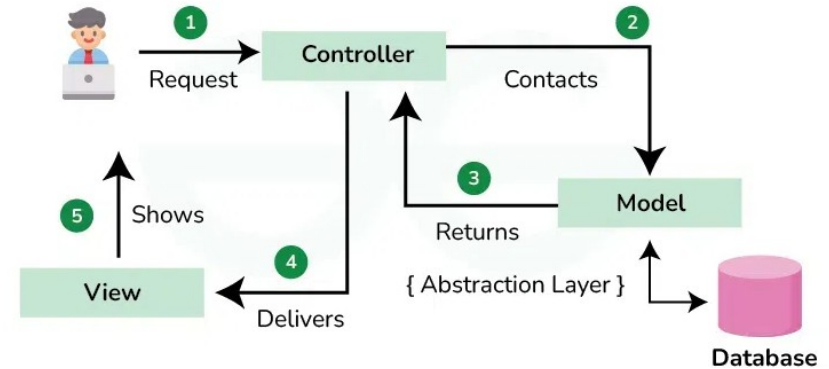
## 2. View (UI)

The View is the visual representation of the Model. It's what the user sees on their screen (HTML, CSS, JSON, or a mobile UI).

**Responsibility:** Presenting data to the user and sending user actions (clicks, form submits) to the Controller.

**Rule:** The View should be "dumb"—it contains very little logic. It just displays what it's told to display.

**Example:** An HTML page with a "Sign Up" button and a text field for an email.



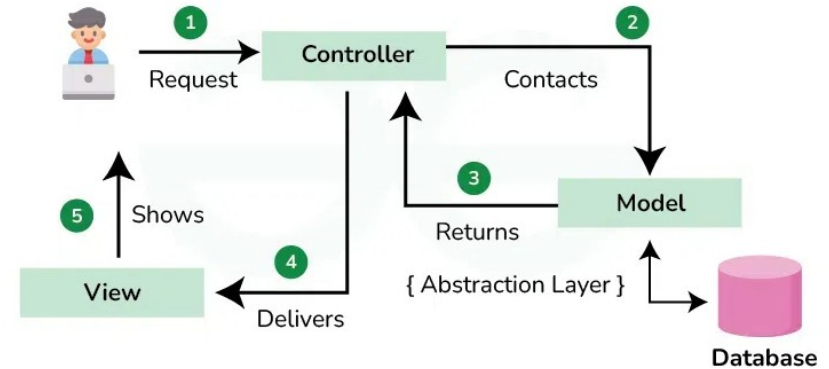
# Model View Controller

## 3. Controller

The Controller acts as the interface between the Model and the View. It processes incoming requests, manipulates data using the Model, and interacts with the View to render the final output.

**Responsibility:** Receiving user input, validating it, asking the Model to update, and deciding which View to show next.

**Example:** A function that takes a "Submit" click, checks if the email is valid, tells the Model to save the user, and then redirects the user to a "Welcome" page.

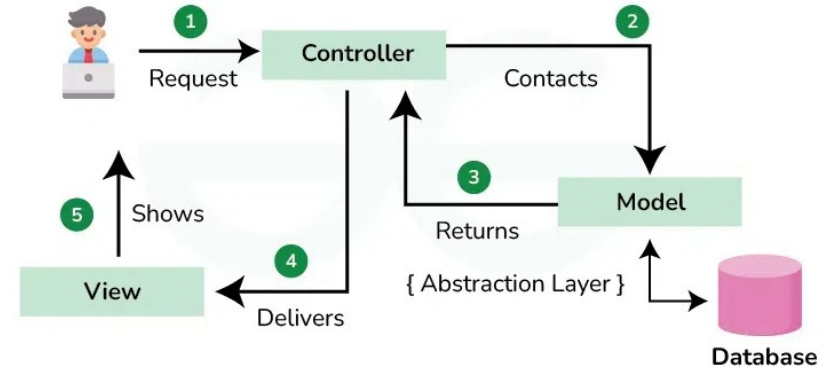


# Model View Controller

## Workflow: Life Cycle

**Example:** User wants to change their profile name:

- **User Action:** The user types a new name in the View and clicks "Save."
- **Request:** The Controller receives this click/request.
- **Update:** The Controller tells the Model to update the name in the database.
- **Response:** The Model confirms the update was successful.
- **Render:** The Controller tells the View to refresh and show a "Profile Updated" message.



# Model View Controller

## Web Application

**Example:** Login System

**Step-1:** User Enter Credentials

Name:-----

password: -----

**Step-2:** Controller receives request

login(username, password)

**Step-3:** Controller calls Model

Model checks database:

```
SELECT * FROM users WHERE username= ...., , ,
```

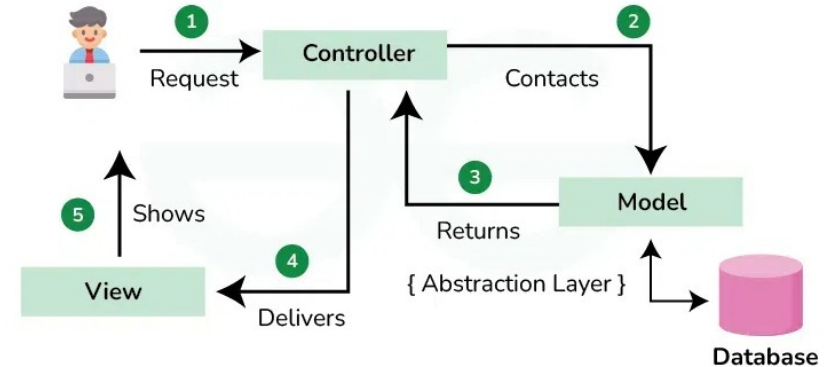
**Step 4 – Model returns result**

valid / invalid

**Step 5 – Controller selects View**

If valid → dashboard page

If invalid → error page



# Model View Controller

## Advantages:

### Separation of Concerns

Each component has clear responsibility.

### Easier Maintenance

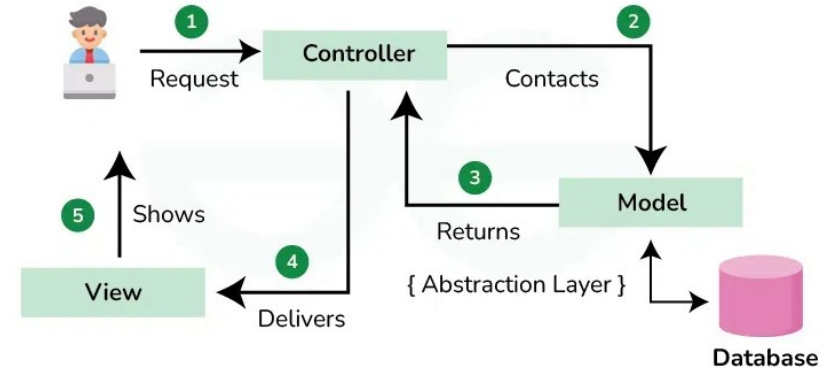
UI changes do not affect data logic.

### Parallel Development

Teams can work independently.

### Reusability

Model can be reused in multiple views.  
Web View, Mobile View,





# Command Query Responsibility Segregation

The Command Query Responsibility Segregation (CQRS) is an architectural pattern that separates the operations that modify data (commands) from the operations that read data (queries).

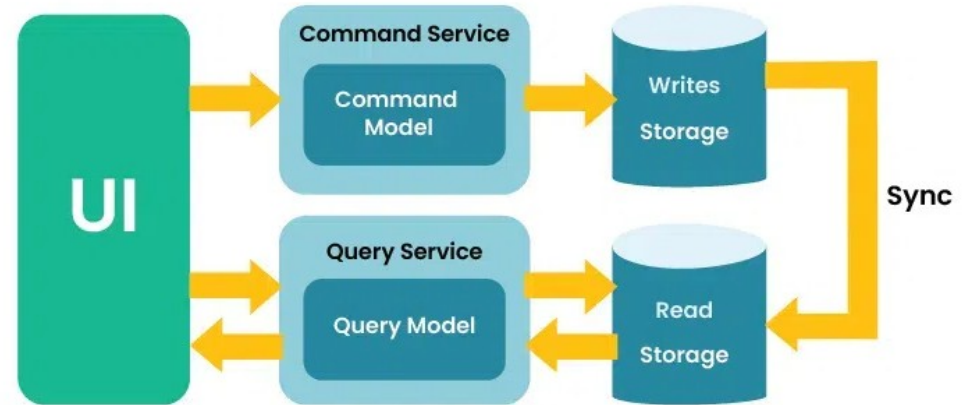
It is widely used in high-scalability and distributed systems to improve performance, scalability, and maintainability.

Traditional systems use one model for both reading and writing data, but CQRS splits them into two separate models.

## Core Idea:

**Commands** → change system state.

**Queries** → read system state.



# Command Query Responsibility Segregation

## Command vs Query:

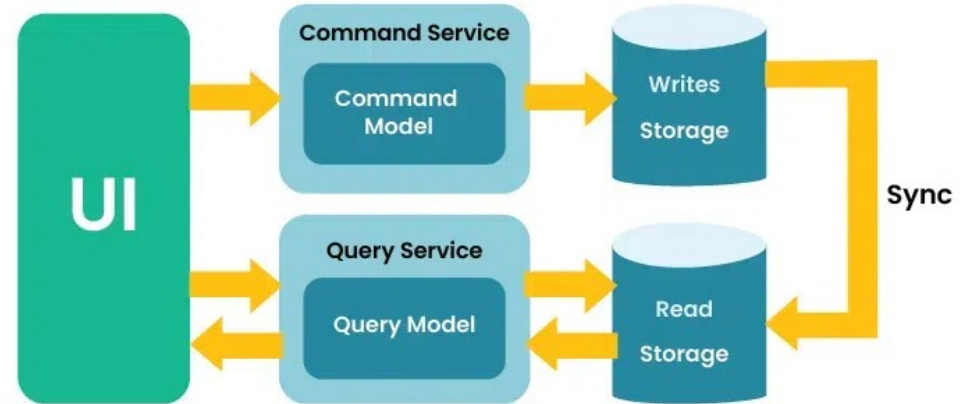
A command performs an action that changes system state.

### Examples:

- Create order
- Update account
- Delete record
- Transfer money

### Characteristics:

- Does not return data
- Only returns success/failure



# Command Query Responsibility Segregation

## Command vs Query:

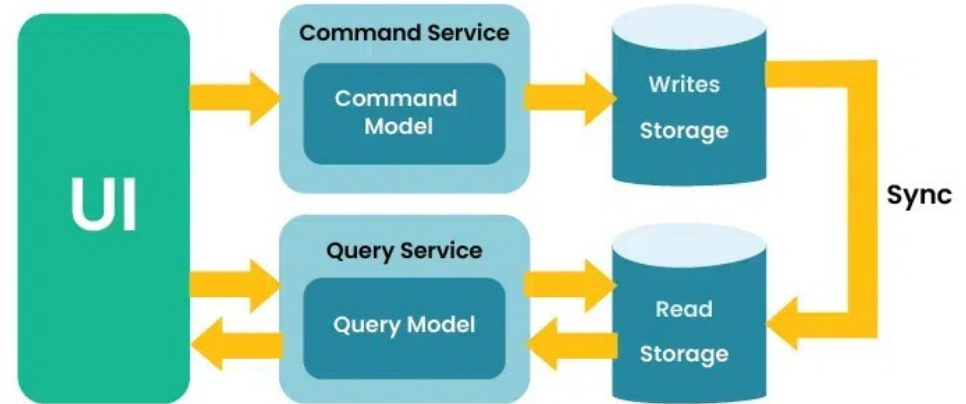
A query retrieves data without modifying it.

### Examples:

- Get user profile
- Get order history
- View account balance

### Characteristics:

- Returns data
- Does not modify system state



# Command Query Responsibility Segregation

## Advantages:

### High Performance

Read operations can be optimized separately.

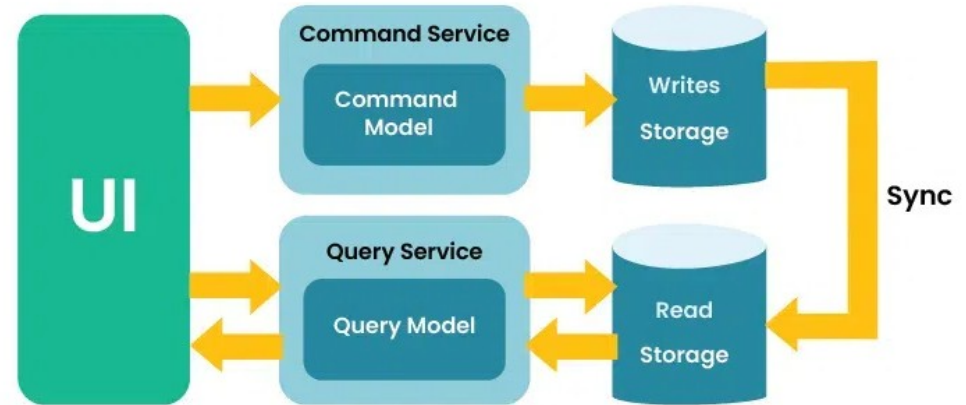
### Scalability

Read and write sides scale independently.

### Flexible Data Models

Read model may use:

- Denormalized data
- Aggregated data



Have A Good Day!